

UNIVERSITAS GUNADARMA
FAKULTAS TEKNOLOGI INDUSTRI



**Analisis Kompresi Gambar Menggunakan Algoritma *JPEG*
dengan *Python Pillow***

Disusun Oleh:

Nama : Muhamad Zidan Indratama
NPM : 50422968
Program Studi : Informatika
Mata Kuliah : Sistem Multimedia
Dosen Pengampu : Dr. Aviarini Indrati., SKom., MM.

Diajukan Guna Melengkapi Sebagian Syarat
Dalam Penilaian Ujian Akhir Semester Mata Kuliah

Jakarta

2026

KATA PENGANTAR

Dengan mengucapkan segala puji dan syukur atas kehadiran Allah SWT yang telah memberikan rahmat dan karunia-Nya sehingga saya dapat menyelesaikan laporan yang berjudul “Analisis Kompresi Gambar Menggunakan Algoritma *JPEG* dengan *Python Pillow*”. Adapun tujuan dari laporan ini adalah sebagai salah satu syarat untuk penilaian ujian akhir semester mata kuliah Sistem Multimedia, Jurusan Informatika, Universitas Gunadarma.

Dalam kesempatan ini saya ingin menyampaikan ucapan terima kasih atas dukungan dan bantuan yang saya terima, sehingga dapat menyelesaikan penulisan ini, dan perkenalkan saya mengucapkan terima kasih kepada:

1. Prof. Dr. E. S. Margianti, SE., MM., selaku Rektor Universitas Gunadarma.
2. Prof. Dr. Ing. Adang Suhendra, SSi., S.Kom., MSc., selaku Dekan Fakultas Teknologi Industri Universitas Gunadarma.
3. Prof. Dr. Lintang Yuniar Banowosari, S.Kom., MSc., selaku Ketua Program Studi Informatika Universitas Gunadarma.
4. Ibu Dr. Aviarini Indrati., SKom., MM., selaku dosen pengampu yang telah memberikan banyak masukan, arahan kepada saya mulai dari awal sampai selesainya laporan ini.
5. Bapak Nusa Indria dan Ibu Linda Sesmita, selaku orang tua tercinta yang selalu mendukung dan terus memberikan motivasi kepada penulis.
6. Seluruh rekan seperjuangan di Universitas Gunadarma yang telah banyak membantu penulis.
7. Semua pihak yang tidak disebutkan yang telah membantu penyelesaian laporan ini, saya ucapkan juga terima kasih atas segala bantuan dan sarannya.

Karena keterbatasan ilmu dan pengalaman yang dimiliki, saya mengharapkan kritik dan saran yang membangun dari pembaca tulisan ini. Saya juga berharap penulisan

ini dapat bermanfaat dan menambah wawasan pembaca dan lebih-lebih pembaca mampu menghasilkan inovasi-inovasi yang baru dari penulisan ini.

Semoga apa yang ada pada penulisan ini dapat bermanfaat bagi kita semua. Aamiin.

Jakarta, 07 Juli 2026

(Muhamad Zidan Indratama)

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI.....	iv
DAFTAR TABEL.....	vii
DAFTAR GAMBAR	viii
DAFTAR LAMPIRAN	ix
1. PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Batasan Masalah.....	3
1.4 Tujuan Penulisan	4
1.5 Manfaat Penulisan	5
2. TINJAUAN PUSTAKA	6
2.1 Kompresi Data.....	6
2.1.1 Definisi Kompresi Data.....	6
2.1.2 Tujuan dan Manfaat Kompresi.....	7
2.1.3 Pengukuran Efektivitas Kompresi.....	8
2.2 Jenis-Jenis Kompresi	10
2.2.1 Kompresi <i>Lossless</i>	11
2.2.2 Kompresi <i>Lossy</i>	12
2.2.3 Perbandingan <i>Lossless</i> dan <i>Lossy</i>	13
2.3 Kompresi Gambar Digital	14
2.3.1 Representasi Gambar Digital	15
2.3.2 Format Gambar Digital	16

2.3.3	Kebutuhan Kompresi pada Gambar Digital	17
2.4	Algoritma Kompresi <i>JPEG</i>	18
2.4.1	Konversi Ruang Warna <i>RGB</i> ke <i>YCbCr</i>	19
2.4.2	<i>Discrete Cosine Transform (DCT)</i>	20
2.4.3	<i>Quantization</i>	21
2.4.4	<i>Entropy Coding</i>	22
2.5	<i>Python Pillow</i>	24
2.5.1	Sekilas tentang <i>Python Pillow</i>	24
2.5.2	Fitur <i>Pillow</i> untuk Kompresi Gambar.....	25
2.5.3	Parameter <i>Quality</i> pada <i>JPEG</i>	26
3.	RANCANGAN DAN IMPLEMENTASI.....	28
3.1	Gambaran Umum Program	28
3.1.1	Tujuan Program.....	28
3.1.2	Bahasa Pemrograman dan <i>Library</i> yang Digunakan	29
3.1.3	Jenis Kompresi yang Digunakan.....	30
3.2	Alur Kerja Program	31
3.2.1	<i>Input</i> Gambar	32
3.2.2	Proses Konversi dan Kompresi	33
3.2.3	Penyimpanan Hasil Kompresi.....	34
3.2.4	Perhitungan Hasil Pengujian	35
3.3	Struktur <i>Folder Project</i>	36
3.3.1	<i>Folder Input</i> Gambar.....	37
3.3.2	<i>Folder Output</i> Gambar Hasil Kompresi.....	38
3.3.3	<i>Folder</i> Hasil Pengujian	39
3.3.4	<i>File Notebook</i> dan <i>File</i> Pendukung.....	39

3.4	Implementasi Program	40
3.4.1	<i>Import Library</i>	41
3.4.2	Penentuan <i>Path Folder</i>	42
3.4.3	Fungsi Kompresi Gambar	43
3.4.4	Fungsi Perhitungan <i>MSE</i> dan <i>PSNR</i>	44
3.4.5	Proses <i>Batch Compression</i>	45
3.5	Data dan Parameter Pengujian	46
3.5.1	Data Gambar Uji	47
3.5.2	Parameter <i>Quality JPEG</i>	47
3.5.3	Format <i>Output</i> Kompresi	48
3.6	Metrik Evaluasi	49
3.6.1	Rasio Kompresi.....	50
3.6.2	<i>Mean Squared Error (MSE)</i>	51
3.6.3	<i>Peak Signal-to-Noise Ratio (PSNR)</i>	52
3.7	<i>Output</i> Program	53
3.7.1	Gambar Hasil Kompresi.....	54
3.7.2	Tabel Hasil Pengujian	56
3.7.3	<i>Preview</i> Perbandingan Gambar.....	57
4.	PENUTUP.....	60
4.1	Kesimpulan.....	60
4.2	Saran	61
	DAFTAR PUSTAKA	62
	LAMPIRAN.....	65

DAFTAR TABEL

Tabel 2.1 Pengukuran Efektivitas Kompresi	9
Tabel 2.2 Perbandingan Kompresi <i>Lossless</i> dan <i>Lossy</i>	13
Tabel 2.3 Perbandingan Format Gambar Digital	17
Tabel 2.4 Tahapan Umum Algoritma Kompresi <i>JPEG</i>	23
Tabel 3.1 Daftar <i>File</i> Gambar dan Ukuran	40
Tabel 3.2 Data Gambar Uji	47
Tabel 3.3 Perbandingan Ukuran <i>Input</i> dan <i>Output</i>	49
Tabel 3.4 Hasil Metrik Evaluasi Kompresi.....	52
Tabel 3.5 Ringkasan <i>Output</i> Program.....	54
Tabel 3.6 Data Gambar <i>Output</i> Hasil Kompresi.....	55

DAFTAR GAMBAR

Gambar 3.1 Daftar <i>Library</i> yang Digunakan	30
Gambar 3.2 Pemindaian Gambar Masukan	32
Gambar 3.3 Proses Konversi dan Kompresi Gambar	33
Gambar 3.4 Penyimpanan Hasil Pengujian.....	34
Gambar 3.5 Struktur <i>Folder Project</i>	37
Gambar 3.6 <i>Import Library</i>	41
Gambar 3.7 Konfigurasi <i>Path Folder</i>	42
Gambar 3.8 Fungsi Konversi <i>RGB</i> dan Kompresi <i>JPEG</i>	43
Gambar 3.9 Struktur Hasil Fungsi Kompresi.....	44
Gambar 3.10 Fungsi Perhitungan <i>MSE</i> dan <i>PSNR</i>	45
Gambar 3.11 Proses <i>Batch Compression</i>	46
Gambar 3.12 Parameter <i>Quality JPEG</i>	48
Gambar 3.13 Perhitungan Rasio Kompresi.....	50
Gambar 3.14 Perhitungan <i>MSE</i> dan <i>PSNR</i>	51
Gambar 3.15 Penyimpanan Gambar Hasil Kompresi	55
Gambar 3.16 Pembuatan Tabel Hasil Pengujian.....	56
Gambar 3.17 <i>Export</i> Tabel Hasil Pengujian ke <i>CSV</i> dan <i>Markdown</i>	57
Gambar 3.18 Visualisasi Perbandingan Gambar	58
Gambar 3.19 <i>Preview</i> Perbandingan Gambar dari <i>Jupyter Notebook</i>	59

DAFTAR LAMPIRAN

Lampiran 1 <i>Setup</i> Kode.....	65
Lampiran 2 Fungsi Kompresi.....	66
Lampiran 3 Proses Pengujian.....	67
Lampiran 4 Tabel Hasil.....	67
Lampiran 5 <i>Preview</i> Visual.....	68
Lampiran 6 <i>Export</i> Hasil.....	68
Lampiran 7 <i>Tautan Source Code Project</i>	68

1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong penggunaan gambar digital dalam berbagai aktivitas, mulai dari dokumentasi, media sosial, situs *web*, aplikasi, hingga kebutuhan pertukaran informasi secara daring. Gambar *digital* menjadi salah satu bentuk data yang banyak digunakan karena mampu menyampaikan informasi secara visual dengan lebih jelas dibandingkan teks. Namun, semakin tinggi kualitas dan resolusi gambar, semakin besar pula ukuran *file* yang dihasilkan.

Ukuran *file* gambar yang besar dapat menimbulkan beberapa kendala, terutama pada aspek penyimpanan dan pengiriman data. Pada media penyimpanan, gambar berukuran besar membutuhkan kapasitas yang lebih banyak. Sementara itu, pada proses pengiriman melalui internet, ukuran *file* yang besar dapat memperlambat proses *upload*, *download*, dan meningkatkan penggunaan *bandwidth*. Kondisi tersebut menunjukkan bahwa efisiensi ukuran *file* menjadi aspek penting dalam pengelolaan gambar digital.

Salah satu cara untuk mengurangi ukuran *file* gambar adalah dengan melakukan kompresi. Kompresi gambar bertujuan untuk memperkecil ukuran *file* agar lebih efisien disimpan dan dikirimkan. Namun, proses kompresi tidak hanya berkaitan dengan pengurangan ukuran *file*, tetapi juga perlu mempertimbangkan kualitas visual gambar setelah dikompresi. Gambar yang terlalu banyak dikompresi dapat mengalami penurunan kualitas, seperti munculnya blok, detail yang berkurang, atau warna yang tidak lagi terlihat halus.

JPEG merupakan salah satu format dan algoritma kompresi gambar yang banyak digunakan karena mampu menghasilkan ukuran *file* yang lebih kecil dengan kualitas visual yang masih dapat diterima. Kompresi *JPEG* termasuk dalam jenis *lossy compression*, yaitu kompresi yang mengurangi sebagian informasi gambar untuk memperoleh ukuran *file* yang lebih kecil. Oleh karena itu, terdapat hubungan kompromi antara ukuran *file* dan kualitas gambar. Semakin tinggi tingkat kompresi

yang diterapkan, semakin kecil ukuran *file* yang dihasilkan, tetapi kualitas visual gambar dapat mengalami penurunan.

Dalam proses analisis kompresi gambar, pustaka *Python Pillow* dapat digunakan sebagai alat bantu untuk membaca, mengolah, mengompresi, dan menyimpan gambar dalam berbagai format. *Python Pillow* memungkinkan pengguna mengatur nilai kualitas kompresi *JPEG*, sehingga hasil kompresi dapat dibandingkan berdasarkan perubahan ukuran *file* dan tampilan visual gambar. Dengan menggunakan pustaka tersebut, proses analisis kompresi gambar dapat dilakukan secara lebih sederhana dan terstruktur.

Berdasarkan uraian tersebut, analisis kompresi gambar menggunakan algoritma *JPEG* dengan *Python Pillow* penting dilakukan untuk memahami pengaruh perubahan nilai kualitas kompresi terhadap ukuran *file* dan kualitas visual gambar. Laporan ini disusun untuk membahas proses kompresi gambar, membandingkan hasil sebelum dan sesudah kompresi, serta mengidentifikasi tingkat kompresi yang dapat menghasilkan ukuran *file* lebih kecil dengan kualitas visual yang masih layak.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, kompresi gambar memiliki peran penting dalam mengurangi ukuran *file* tanpa mengabaikan kualitas visual secara berlebihan. Penggunaan algoritma *JPEG* dan pustaka *Python Pillow* memungkinkan proses kompresi dilakukan dengan pengaturan nilai kualitas tertentu. Oleh karena itu, diperlukan rumusan masalah yang dapat mengarahkan pembahasan agar laporan ini tetap terfokus.

Rumusan masalah dalam laporan ini adalah sebagai berikut:

1. Bagaimana proses kompresi gambar menggunakan algoritma *JPEG* dengan pustaka *Python Pillow*?
2. Bagaimana pengaruh perubahan nilai kualitas kompresi terhadap ukuran *file* gambar?

3. Bagaimana perbandingan kualitas visual gambar sebelum dan sesudah proses kompresi?
4. Bagaimana menentukan tingkat kompresi yang menghasilkan ukuran *file* lebih kecil dengan kualitas visual yang masih layak?

Rumusan masalah tersebut menjadi dasar dalam penyusunan pembahasan laporan. Dengan adanya rumusan masalah yang jelas, analisis dapat difokuskan pada hubungan antara nilai kualitas kompresi, ukuran *file* hasil kompresi, dan perubahan kualitas visual gambar yang dihasilkan.

1.3 Batasan Masalah

Agar pembahasan dalam laporan ini tidak terlalu luas, diperlukan batasan masalah yang menjelaskan ruang lingkup analisis. Batasan ini digunakan agar laporan tetap berfokus pada kompresi gambar menggunakan algoritma *JPEG* dengan bantuan pustaka *Python Pillow*. Dengan demikian, pembahasan tidak melebar ke metode kompresi atau pengolahan citra lain yang berada di luar topik utama.

Batasan masalah dalam laporan ini adalah sebagai berikut:

1. Laporan hanya membahas kompresi gambar menggunakan algoritma *JPEG*.
2. Implementasi analisis dilakukan menggunakan bahasa pemrograman *Python* dan pustaka *Python Pillow*.
3. Gambar yang dianalisis berupa gambar digital statis, bukan video atau animasi.
4. Pembahasan difokuskan pada perubahan ukuran *file* dan kualitas visual hasil kompresi.
5. Format keluaran gambar difokuskan pada format *JPEG*, *.jpg*, atau *.jpeg*.
6. Laporan tidak membahas kompresi gambar berbasis *machine learning*.
7. Laporan tidak membahas secara mendalam algoritma kompresi lain seperti *PNG*, *WebP*, *GIF*, atau *HEIF*.

8. Laporan tidak membahas keamanan gambar, enkripsi, atau proses transmisi jaringan secara mendalam.

Dengan batasan tersebut, laporan ini diarahkan pada analisis sederhana mengenai kompresi gambar menggunakan *JPEG* dan *Python Pillow*. Fokus utama pembahasan berada pada proses kompresi, perubahan ukuran *file*, serta pengamatan kualitas gambar sebelum dan sesudah dikompresi.

1.4 Tujuan Penulisan

Tujuan penulisan disusun untuk menjawab rumusan masalah yang telah ditetapkan. Dalam laporan ini, tujuan penulisan diarahkan pada pemahaman proses kompresi gambar serta analisis pengaruh nilai kualitas kompresi terhadap ukuran *file* dan tampilan visual gambar. Tujuan tersebut disusun agar pembahasan memiliki arah yang jelas dan dapat disampaikan secara sistematis.

Tujuan penulisan laporan ini adalah sebagai berikut:

1. Menjelaskan proses kompresi gambar menggunakan algoritma *JPEG* dengan pustaka *Python Pillow*.
2. Menganalisis pengaruh nilai kualitas kompresi terhadap ukuran *file* gambar.
3. Membandingkan kualitas visual gambar sebelum dan sesudah proses kompresi.
4. Mengidentifikasi tingkat kompresi yang menghasilkan ukuran *file* lebih kecil dengan kualitas visual yang masih layak.

Melalui tujuan tersebut, laporan ini diharapkan dapat memberikan gambaran mengenai hubungan antara kompresi, ukuran *file*, dan kualitas gambar. Pembahasan pada laporan ini tidak diarahkan untuk menciptakan algoritma kompresi baru, melainkan untuk memahami dan menganalisis penerapan kompresi *JPEG* menggunakan *Python Pillow*.

1.5 Manfaat Penulisan

Penulisan laporan ini diharapkan dapat memberikan manfaat bagi penulis maupun pembaca. Manfaat yang dimaksud berkaitan dengan pemahaman konsep kompresi gambar, penerapan pustaka *Python Pillow*, serta kemampuan menganalisis hasil kompresi berdasarkan ukuran *file* dan kualitas visual. Dengan demikian, laporan ini dapat digunakan sebagai bahan pembelajaran dasar dalam pengolahan gambar digital.

Manfaat penulisan laporan ini adalah sebagai berikut:

1. Bagi penulis, laporan ini menjadi sarana untuk memahami konsep kompresi gambar dan penerapannya menggunakan *Python Pillow*.
2. Bagi pembaca, laporan ini dapat menjadi referensi dasar mengenai pengaruh kompresi *JPEG* terhadap ukuran *file* dan kualitas gambar.
3. Bagi pembelajaran, laporan ini dapat menjadi contoh sederhana penerapan pengolahan citra digital menggunakan *Python*.
4. Bagi pengembangan laporan selanjutnya, laporan ini dapat menjadi dasar untuk membandingkan metode kompresi gambar lain.

Manfaat tersebut disusun secara realistis sesuai dengan ruang lingkup laporan. Dengan adanya pembahasan ini, pembaca diharapkan dapat memahami bahwa kompresi gambar tidak hanya berkaitan dengan memperkecil ukuran *file*, tetapi juga perlu mempertimbangkan kualitas visual yang dihasilkan setelah proses kompresi.

2. TINJAUAN PUSTAKA

2.1 Kompresi Data

Kompresi data merupakan konsep dasar dalam pengelolaan data digital yang bertujuan mengurangi ukuran data agar lebih efisien untuk disimpan, diproses, maupun dikirimkan. Secara umum, kompresi bekerja dengan merepresentasikan informasi menggunakan jumlah bit yang lebih sedikit dibandingkan bentuk aslinya. Proses ini dapat dilakukan dengan mengurangi redundansi, menyederhanakan representasi data, atau menghilangkan sebagian informasi yang dianggap kurang berpengaruh terhadap persepsi pengguna (Salomon, 2007).

Pada konteks data digital, kebutuhan kompresi muncul karena ukuran *file* yang besar dapat memengaruhi efisiensi penyimpanan dan pengiriman. IBM menjelaskan bahwa kompresi data digunakan untuk mengurangi ukuran kumpulan data dengan menghilangkan elemen yang berulang, serta banyak diterapkan pada penyimpanan cadangan, transmisi jaringan, dan penyimpanan awan (IBM, 2024). Dengan demikian, kompresi tidak hanya berkaitan dengan penghematan ruang penyimpanan, tetapi juga berhubungan dengan efisiensi akses dan distribusi data.

Dalam pengolahan gambar digital, kompresi data menjadi dasar penting sebelum membahas format *JPEG* secara lebih khusus. Gambar digital biasanya tersusun dari banyak *pixel* dan dapat menghasilkan ukuran *file* yang besar, terutama ketika memiliki resolusi tinggi atau kedalaman warna yang besar. Oleh karena itu, pemahaman mengenai definisi, tujuan, manfaat, serta pengukuran efektivitas kompresi diperlukan sebagai landasan untuk menganalisis hasil kompresi gambar menggunakan *Python Pillow*.

2.1.1 Definisi Kompresi Data

Kompresi data dapat didefinisikan sebagai proses pengkodean informasi agar membutuhkan jumlah bit yang lebih sedikit dibandingkan representasi awalnya. Salomon menjelaskan bahwa kompresi data berkaitan dengan upaya mengurangi jumlah data yang diperlukan untuk merepresentasikan suatu informasi, baik melalui

pendekatan yang mempertahankan seluruh informasi maupun pendekatan yang mengorbankan sebagian informasi tertentu (Salomon, 2007). Dengan kata lain, kompresi bertujuan membuat data menjadi lebih ringkas tanpa menghilangkan fungsi utama data tersebut.

Prinsip utama kompresi adalah memanfaatkan redundansi yang terdapat dalam data. Redundansi dapat berupa pola berulang, nilai yang sering muncul, atau bagian data yang dapat direpresentasikan dengan cara lebih sederhana. Pada gambar digital, redundansi dapat muncul dalam bentuk area warna yang serupa, pola tekstur yang berulang, atau informasi visual yang tidak terlalu sensitif terhadap pengamatan manusia. IBM menyebutkan bahwa kompresi mengurangi ukuran data dengan menghilangkan elemen-elemen repetitif, sehingga data dapat disimpan dan ditransmisikan secara lebih efisien (IBM, 2024).

Kompresi data dapat diterapkan pada berbagai bentuk data, seperti teks, audio, video, dan gambar. Pada laporan ini, fokus pembahasan diarahkan pada kompresi gambar digital menggunakan algoritma *JPEG*. Oleh karena itu, definisi kompresi data digunakan sebagai dasar untuk memahami bagaimana sebuah gambar dapat disimpan dalam ukuran *file* yang lebih kecil, sekaligus tetap mempertahankan kualitas visual yang masih dapat diterima.

2.1.2 Tujuan dan Manfaat Kompresi

Tujuan utama kompresi adalah mengurangi ukuran data agar penggunaan ruang penyimpanan menjadi lebih efisien. *File* yang lebih kecil membutuhkan kapasitas penyimpanan yang lebih sedikit, sehingga lebih mudah dikelola dalam jumlah besar. Hal ini penting pada pengelolaan gambar digital karena satu gambar dengan resolusi tinggi dapat memiliki ukuran *file* yang cukup besar, terutama jika disimpan tanpa kompresi atau dengan format yang kurang efisien.

Selain menghemat penyimpanan, kompresi juga bermanfaat dalam mempercepat proses pengiriman data. Pada lingkungan digital, *file* yang lebih kecil umumnya lebih cepat diunggah, diunduh, dan dibagikan melalui jaringan. IBM menjelaskan bahwa kompresi banyak digunakan pada transmisi jaringan dan penyimpanan awan

untuk mengoptimalkan ruang penyimpanan serta meningkatkan efisiensi pengambilan data (IBM, 2024). Dengan demikian, kompresi berperan penting dalam meningkatkan efisiensi penggunaan data pada aplikasi, situs web, dan sistem berbasis internet.

Pada konteks kompresi gambar, manfaat kompresi perlu dipahami secara seimbang. Ukuran *file* yang lebih kecil memang memberikan keuntungan dari sisi penyimpanan dan pengiriman, tetapi kualitas visual gambar tetap harus diperhatikan. Khusus pada format *JPEG*, proses kompresi dapat menghasilkan ukuran *file* yang lebih kecil dengan konsekuensi penurunan kualitas tertentu karena format ini termasuk kompresi *lossy*. Oleh karena itu, manfaat kompresi tidak hanya diukur dari seberapa kecil ukuran *file* yang dihasilkan, tetapi juga dari sejauh mana gambar hasil kompresi masih layak digunakan secara visual.

2.1.3 Pengukuran Efektivitas Kompresi

Efektivitas kompresi dapat dinilai dari dua sisi utama, yaitu pengurangan ukuran *file* dan perubahan kualitas data setelah proses kompresi. Pada kompresi gambar, ukuran *file* yang lebih kecil menunjukkan bahwa kompresi berhasil menghemat ruang penyimpanan. Namun, pengurangan ukuran *file* tidak selalu berarti hasil kompresi baik, karena kualitas gambar dapat menurun apabila informasi visual yang dihilangkan terlalu besar. Oleh karena itu, evaluasi kompresi gambar umumnya memperhatikan rasio kompresi serta metrik kualitas seperti *MSE* dan *PSNR*.

Rasio kompresi digunakan untuk membandingkan ukuran data sebelum dan sesudah kompresi. Secara sederhana, rasio kompresi dapat dipahami sebagai perbandingan antara ukuran *file* asli dan ukuran *file* hasil kompresi. Jika ukuran *file* hasil kompresi jauh lebih kecil, maka kompresi dianggap lebih efisien dari sisi penyimpanan. Namun, rasio kompresi hanya menjelaskan pengurangan ukuran *file* dan belum menunjukkan tingkat kemiripan antara gambar asli dan gambar hasil kompresi.

Untuk menilai perbedaan antara gambar asli dan gambar hasil kompresi, metrik *MSE* atau *Mean Squared Error* dapat digunakan. *MSE* merepresentasikan rata-rata kuadrat selisih nilai *pixel* antara gambar asli dan gambar hasil kompresi. MathWorks menjelaskan bahwa *MSE* menunjukkan galat kuadrat kumulatif antara gambar terkompresi dan gambar asli, sedangkan *PSNR* digunakan sebagai ukuran galat puncak (MathWorks, 2024). Nilai *MSE* yang lebih kecil menunjukkan bahwa perbedaan antara kedua gambar relatif lebih rendah.

Selain *MSE*, metrik *PSNR* atau *Peak Signal-to-Noise Ratio* juga sering digunakan untuk mengevaluasi kualitas gambar hasil kompresi. *PSNR* menunjukkan perbandingan antara nilai maksimum sinyal gambar dan tingkat gangguan atau galat yang muncul setelah proses kompresi. Wang, Zheng, Chen, dan Principe menjelaskan bahwa *MSE* dan *PSNR* termasuk metrik tradisional yang banyak digunakan dalam penilaian kualitas gambar karena berbasis pada perbedaan nilai absolut antar gambar (Wang et al., 2017). Secara umum, nilai *PSNR* yang lebih tinggi menunjukkan kualitas gambar yang lebih baik secara kuantitatif.

Tabel 2.1 Pengukuran Efektivitas Kompresi

Metrik	Fungsi	Interpretasi Umum
Rasio kompresi	Membandingkan ukuran <i>file</i> asli dan ukuran <i>file</i> hasil kompresi.	Semakin besar pengurangan ukuran <i>file</i> , semakin efisien kompresi dari sisi penyimpanan.
<i>MSE</i>	Mengukur rata-rata kuadrat perbedaan nilai <i>pixel</i> antara gambar asli dan gambar hasil kompresi.	Semakin kecil nilai <i>MSE</i> , semakin kecil perbedaan antara gambar asli dan hasil kompresi.
<i>PSNR</i>	Mengukur kualitas gambar berdasarkan perbandingan sinyal maksimum dan galat kompresi.	Semakin tinggi nilai <i>PSNR</i> , semakin baik kualitas gambar secara kuantitatif.

Dalam analisis kompresi gambar, ketiga ukuran tersebut dapat digunakan secara saling melengkapi. Rasio kompresi menunjukkan efisiensi ukuran *file*, sedangkan *MSE* dan *PSNR* membantu memberikan gambaran kuantitatif mengenai perubahan kualitas gambar. Untuk kebutuhan laporan ini, pemahaman terhadap metrik tersebut menjadi dasar dalam membahas pengaruh pengaturan kualitas kompresi *JPEG* menggunakan *Python Pillow* pada Bab 3 Rancangan dan Implementasi. Struktur dan batasan subbab ini mengikuti arahan laporan yang telah diberikan sebelumnya.

2.2 Jenis-Jenis Kompresi

Kompresi data dapat dibedakan berdasarkan cara proses kompresi memperlakukan informasi asli. Secara umum, terdapat dua pendekatan utama, yaitu kompresi *lossless* dan kompresi *lossy*. Keduanya memiliki tujuan yang sama, yaitu mengurangi ukuran *file*, tetapi berbeda dalam cara menjaga atau mengurangi informasi yang terkandung di dalam data. Perbedaan ini penting karena pemilihan jenis kompresi akan memengaruhi ukuran *file*, kualitas hasil, serta kesesuaian penggunaan data setelah proses kompresi dilakukan.

Pada kompresi gambar digital, perbedaan antara *lossless* dan *lossy* menjadi semakin penting karena gambar tidak hanya dinilai dari ukuran *file*, tetapi juga dari kualitas visualnya. Pangesti, Widagdo, Riana, dan Hadiani menjelaskan bahwa kompresi *lossless* tidak menghilangkan informasi selama proses kompresi, sedangkan kompresi *lossy* menghilangkan sebagian informasi untuk menghasilkan ukuran *file* yang lebih kecil (Endah Pangesti et al., 2029). Dengan demikian, kompresi *lossless* lebih menekankan keutuhan data, sedangkan kompresi *lossy* lebih menekankan efisiensi ukuran *file* dengan tetap mempertimbangkan kualitas yang masih dapat diterima.

Pada laporan ini, jenis kompresi yang digunakan adalah kompresi *lossy* karena analisis difokuskan pada format *JPEG*. Kompresi *lossless* tetap dibahas sebagai landasan perbandingan agar perbedaan pendekatan kompresi dapat dipahami secara lebih jelas. Dengan adanya pembahasan tersebut, posisi *JPEG* sebagai format kompresi gambar yang mengurangi sebagian informasi visual dapat dijelaskan

secara lebih tepat sebelum masuk ke pembahasan algoritma *JPEG* pada subbab berikutnya.

2.2.1 Kompresi *Lossless*

Kompresi *lossless* merupakan teknik kompresi yang memungkinkan data asli dikembalikan secara utuh setelah proses dekompresi. Pada pendekatan ini, tidak ada informasi yang hilang selama proses kompresi berlangsung. MDN Web Docs menjelaskan bahwa *lossless compression* merupakan kelas algoritma kompresi data yang memungkinkan data asli direkonstruksi kembali secara sempurna dari data hasil kompresi (MDN Web Docs, 2025b).

Keunggulan utama kompresi *lossless* terletak pada kemampuan menjaga keutuhan data. Pendekatan ini cocok digunakan untuk data yang tidak boleh berubah, seperti dokumen, arsip, data teknis, citra medis, atau gambar yang membutuhkan ketelitian tinggi. Pada konteks gambar digital, kompresi *lossless* dapat mempertahankan seluruh detail gambar karena tidak ada bagian informasi yang dibuang. Oleh karena itu, hasil dekompresi dari kompresi *lossless* tetap sama dengan data asli.

Meskipun demikian, kompresi *lossless* umumnya menghasilkan pengurangan ukuran *file* yang lebih terbatas dibandingkan kompresi *lossy*. Hal ini terjadi karena seluruh informasi asli tetap harus dipertahankan. Hussain, Al-Fayadh, dan Radi menjelaskan bahwa kompresi gambar *lossless* menuntut gambar hasil dekompresi menjadi replika yang sama dengan gambar asli, sehingga ruang optimasi ukuran *file* lebih terbatas dibandingkan pendekatan yang menghilangkan sebagian informasi (Hussain et al., 2018).

Pada laporan ini, kompresi *lossless* tidak digunakan sebagai metode utama dalam proses analisis. Pembahasan mengenai kompresi *lossless* ditempatkan sebagai pembandingan teori untuk memperjelas perbedaan dengan kompresi *lossy*. Dengan memahami karakteristik *lossless*, pembahasan mengenai *JPEG* sebagai format kompresi *lossy* dapat dipahami dengan lebih terarah.

2.2.2 Kompresi Lossy

Kompresi *lossy* merupakan teknik kompresi yang mengurangi ukuran data dengan menghilangkan sebagian informasi dari data asli. Informasi yang dihilangkan umumnya merupakan bagian yang dianggap kurang berpengaruh terhadap persepsi pengguna. Pada gambar digital, pendekatan ini memanfaatkan keterbatasan penglihatan manusia terhadap detail tertentu, sehingga sebagian informasi visual dapat dikurangi tanpa selalu terlihat secara jelas oleh pengguna.

MDN Web Docs menjelaskan bahwa *lossy compression* adalah metode kompresi data yang menggunakan pendekatan tidak persis dan membuang sebagian data untuk merepresentasikan konten dalam ukuran yang lebih kecil (MDN Web Docs, 2025c). Karena sebagian data dihilangkan, proses kompresi *lossy* bersifat tidak sepenuhnya dapat dibalik. Artinya, gambar hasil kompresi tidak dapat dikembalikan secara identik ke bentuk aslinya setelah proses kompresi dilakukan.

Keunggulan kompresi *lossy* terletak pada kemampuannya menghasilkan ukuran *file* yang lebih kecil dibandingkan pendekatan *lossless*. Namun, pengurangan ukuran *file* tersebut memiliki konsekuensi berupa potensi penurunan kualitas. Pada gambar digital, penurunan kualitas dapat muncul dalam bentuk berkurangnya detail, perubahan warna halus, atau munculnya artefak kompresi. Adobe menjelaskan bahwa kompresi *lossy* menghapus sebagian data secara permanen untuk mengurangi ukuran *file*, sedangkan kompresi *lossless* mempertahankan data agar dapat dipulihkan Kembali (Adobe, 2024).

Pada laporan ini, kompresi *lossy* menjadi fokus utama karena format *JPEG* bekerja dengan pendekatan tersebut. Analisis kompresi gambar menggunakan *Python Pillow* diarahkan untuk melihat bagaimana pengaturan kualitas pada *JPEG* dapat memengaruhi ukuran *file* dan kualitas visual gambar. Dengan demikian, pembahasan kompresi *lossy* menjadi dasar penting sebelum membahas proses kerja algoritma *JPEG* secara lebih rinci.

2.2.3 Perbandingan *Lossless* dan *Lossy*

Perbandingan antara kompresi *lossless* dan *lossy* diperlukan untuk memahami alasan pemilihan pendekatan kompresi pada suatu jenis data. Kedua teknik tersebut sama-sama dapat mengurangi ukuran *file*, tetapi memiliki konsekuensi yang berbeda terhadap keutuhan data. Kompresi *lossless* menjaga agar data asli dapat dikembalikan tanpa perubahan, sedangkan kompresi *lossy* menghilangkan sebagian informasi untuk memperoleh ukuran *file* yang lebih kecil.

Pada pengolahan gambar digital, pemilihan antara *lossless* dan *lossy* bergantung pada kebutuhan penggunaan gambar. Jika gambar harus dipertahankan secara utuh, maka kompresi *lossless* lebih sesuai. Sebaliknya, jika tujuan utama adalah mengurangi ukuran *file* dan sedikit penurunan kualitas masih dapat diterima, maka kompresi *lossy* menjadi pilihan yang lebih relevan. Pangesti, Widagdo, Riana, dan Hadiani menjelaskan bahwa perbedaan mendasar antara kedua metode tersebut terletak pada ada atau tidaknya informasi yang hilang selama proses kompresi (Endah Pangesti et al., 2029).

Perbedaan antara kompresi *lossless* dan *lossy* dapat diringkas pada tabel berikut.

Tabel 2.2 Perbandingan Kompresi *Lossless* dan *Lossy*

Aspek	Kompresi <i>Lossless</i>	Kompresi <i>Lossy</i>
Keutuhan data	Data asli dapat dikembalikan tanpa perubahan.	Sebagian informasi asli dihilangkan.
Kualitas hasil	Kualitas tetap sama dengan data asli.	Kualitas dapat menurun bergantung tingkat kompresi.
Ukuran <i>file</i>	Pengurangan ukuran <i>file</i> biasanya lebih terbatas.	Pengurangan ukuran <i>file</i> umumnya lebih besar.
Proses dekompresi	Data dapat direkonstruksi secara identik.	Data tidak dapat dikembalikan secara identik.

Aspek	Kompresi <i>Lossless</i>	Kompresi <i>Lossy</i>
Kesesuaian penggunaan	Cocok untuk dokumen, arsip, gambar teknis, dan data yang harus akurat.	Cocok untuk foto, gambar web, dan konten visual yang toleran terhadap penurunan kualitas.
Contoh format	<i>PNG</i> , <i>GIF</i> , dan beberapa variasi <i>TIFF</i> .	<i>JPEG</i> dan <i>WebP</i> mode <i>lossy</i> .

Berdasarkan perbandingan tersebut, kompresi *lossy* lebih relevan dengan laporan ini karena format *JPEG* bekerja dengan mengurangi sebagian informasi visual untuk menghasilkan ukuran *file* yang lebih kecil. Kompresi *lossless* tidak digunakan dalam proses analisis, tetapi tetap penting sebagai pembanding teori. Oleh karena itu, pembahasan berikutnya lebih diarahkan pada kompresi gambar digital dan algoritma *JPEG* sebagai pendekatan kompresi yang digunakan dalam laporan ini.

2.3 Kompresi Gambar Digital

Kompresi gambar digital merupakan penerapan kompresi data pada berkas gambar agar ukuran *file* menjadi lebih efisien untuk disimpan, dikirim, dan digunakan pada berbagai media digital. Gambar digital memiliki karakteristik berbeda dibandingkan data teks karena tersusun dari elemen visual seperti *pixel*, warna, resolusi, dan detail visual. Oleh karena itu, proses kompresi gambar perlu mempertimbangkan dua aspek sekaligus, yaitu pengurangan ukuran *file* dan kualitas visual gambar setelah kompresi dilakukan.

Dalam konteks penggunaan gambar pada aplikasi dan situs web, ukuran gambar dapat memengaruhi kinerja akses pengguna. Web.dev menjelaskan bahwa kompresi gambar berperan dalam mengurangi ukuran *file*, tetapi hasil kompresi tetap perlu diperiksa agar kualitasnya sesuai dengan kebutuhan tampilan (Web.dev, 2023). Hal ini menunjukkan bahwa kompresi gambar tidak hanya bertujuan memperkecil *file*, tetapi juga menjaga agar gambar tetap dapat digunakan secara layak.

Pada laporan ini, pembahasan kompresi gambar digital diarahkan untuk memahami dasar representasi gambar, format-format gambar yang umum digunakan, serta alasan mengapa gambar perlu dikompresi. Pemahaman tersebut diperlukan sebelum membahas algoritma *JPEG* secara lebih rinci, karena efektivitas kompresi gambar sangat dipengaruhi oleh karakteristik gambar, format penyimpanan, dan tujuan penggunaannya.

2.3.1 Representasi Gambar Digital

Gambar digital dapat dipahami sebagai representasi visual yang tersusun dari kumpulan *pixel*. Setiap *pixel* menyimpan informasi warna atau intensitas tertentu yang membentuk keseluruhan tampilan gambar. Semakin banyak jumlah *pixel* yang dimiliki sebuah gambar, semakin tinggi pula resolusinya. Resolusi yang lebih tinggi umumnya menghasilkan detail visual yang lebih baik, tetapi juga dapat meningkatkan ukuran *file* karena jumlah data yang harus disimpan menjadi lebih banyak.

Selain resolusi, kedalaman bit atau *bit depth* juga memengaruhi ukuran dan kualitas gambar digital. *Bit depth* menunjukkan jumlah bit yang digunakan untuk merepresentasikan warna atau intensitas pada setiap *pixel*. Evident Scientific menjelaskan bahwa gambar digital warna dipengaruhi oleh konsep seperti *sampling*, *quantization*, resolusi spasial, *bit depth*, dan ruang warna; pada gambar berwarna, *pixel* direpresentasikan melalui beberapa komponen kecerahan warna (Evident Scientific, 2026). Dengan demikian, semakin besar kedalaman bit, semakin banyak variasi warna yang dapat direpresentasikan, tetapi kebutuhan penyimpanan juga meningkat.

Ruang warna atau *color space* menjadi bagian lain yang penting dalam representasi gambar digital. Ruang warna menentukan bagaimana warna direpresentasikan dalam bentuk angka, misalnya *RGB* yang menggunakan komponen merah, hijau, dan biru. Pada proses kompresi tertentu seperti *JPEG*, representasi warna dapat diubah ke ruang warna lain agar informasi luminansi dan krominansi dapat diproses secara lebih efisien. Pemahaman mengenai resolusi, *bit depth*, dan ruang warna

menjadi dasar untuk memahami mengapa gambar dengan karakteristik berbeda dapat menghasilkan ukuran *file* dan kualitas kompresi yang berbeda pula.

Dalam analisis kompresi gambar menggunakan *Python Pillow*, representasi gambar digital menjadi dasar untuk melihat perubahan yang terjadi sebelum dan sesudah kompresi. Gambar dengan resolusi tinggi atau variasi warna yang kompleks umumnya membutuhkan ukuran *file* lebih besar dibandingkan gambar sederhana. Oleh karena itu, karakteristik gambar perlu diperhatikan ketika membandingkan hasil kompresi, terutama pada format *JPEG* yang bekerja dengan pendekatan *lossy compression*.

2.3.2 Format Gambar Digital

Format gambar digital merupakan cara penyimpanan data gambar ke dalam *file* dengan struktur tertentu. Setiap format memiliki karakteristik yang berbeda, baik dari sisi jenis kompresi, dukungan warna, transparansi, maupun kesesuaian penggunaan. MDN Web Docs menjelaskan bahwa pemilihan format gambar perlu mempertimbangkan jenis gambar dan kebutuhan penggunaan, karena setiap format memiliki kelebihan dan keterbatasan masing-masing (MDN Web Docs, 2026).

Beberapa format gambar yang umum digunakan adalah *JPEG*, *PNG*, *BMP*, dan *GIF*. *JPEG* banyak digunakan untuk foto atau gambar dengan gradasi warna kompleks karena mampu menghasilkan ukuran *file* lebih kecil melalui kompresi *lossy*. *PNG* umumnya digunakan untuk gambar yang membutuhkan transparansi dan kompresi *lossless*. *BMP* menyimpan data gambar dengan struktur yang relatif sederhana dan dapat menghasilkan ukuran *file* besar karena tidak selalu menggunakan kompresi yang efisien. Sementara itu, *GIF* dikenal mendukung animasi sederhana dan palet warna terbatas.

Perbedaan format gambar digital dapat diringkas pada tabel berikut.

Tabel 2.3 Perbandingan Format Gambar Digital

Format	Karakteristik Umum	Kesesuaian Penggunaan
<i>JPEG</i>	Menggunakan kompresi <i>lossy</i> dan cocok untuk gambar dengan banyak variasi warna.	Foto, gambar web, dan gambar yang membutuhkan ukuran <i>file</i> lebih kecil.
<i>PNG</i>	Mendukung kompresi <i>lossless</i> dan transparansi.	Logo, ikon, ilustrasi, dan gambar yang membutuhkan detail tajam.
<i>BMP</i>	Struktur penyimpanan sederhana dan ukuran <i>file</i> cenderung besar.	Penyimpanan gambar tanpa kebutuhan kompresi tinggi.
<i>GIF</i>	Mendukung animasi sederhana dan palet warna terbatas.	Animasi ringan, ikon bergerak, atau gambar sederhana.

Tabel tersebut menunjukkan bahwa pemilihan format gambar perlu disesuaikan dengan kebutuhan penggunaan. Pada laporan ini, format yang digunakan adalah *JPEG* karena fokus analisis berada pada kompresi gambar berbasis *lossy compression*. Format lain seperti *PNG*, *BMP*, dan *GIF* dibahas sebagai pembanding agar karakteristik *JPEG* dapat dipahami secara lebih jelas, tetapi tidak digunakan sebagai fokus utama analisis.

2.3.3 Kebutuhan Kompresi pada Gambar Digital

Gambar digital perlu dikompresi karena ukuran *file* yang besar dapat memengaruhi efisiensi penyimpanan dan pengiriman data. Pada perangkat penyimpanan, gambar berukuran besar membutuhkan kapasitas yang lebih banyak. Pada proses distribusi melalui internet, *file* gambar yang besar dapat memperlambat proses *upload*, *download*, dan meningkatkan penggunaan *bandwidth*. Oleh karena itu, kompresi menjadi salah satu langkah penting untuk membuat penggunaan gambar digital menjadi lebih efisien.

Kebutuhan kompresi semakin terlihat pada aplikasi dan situs web yang menggunakan banyak gambar. Cloudinary menjelaskan bahwa optimasi gambar

melalui kompresi dan pemilihan format yang tepat dapat membantu mempercepat waktu muat halaman, mengurangi kebutuhan penyimpanan, serta menghemat *bandwidth* (Cloudinary, 2020). Dengan ukuran *file* yang lebih kecil, gambar dapat ditampilkan lebih cepat dan penggunaan sumber daya menjadi lebih ringan.

Pada gambar digital, kompresi juga diperlukan untuk menyesuaikan kualitas visual dengan kebutuhan penggunaan. Tidak semua gambar harus disimpan dalam kualitas tertinggi, terutama jika gambar tersebut digunakan untuk tampilan web, pratinjau, atau kebutuhan dokumentasi ringan. Namun, proses kompresi tetap perlu dikendalikan agar penurunan kualitas tidak terlalu mengganggu. Web.dev menekankan bahwa pada kompresi *lossy*, hasil kompresi perlu diperiksa agar tetap memenuhi standar kualitas yang diinginkan (Web.dev, 2023).

Untuk laporan ini, kebutuhan kompresi gambar digital berkaitan langsung dengan analisis penggunaan format *JPEG* melalui *Python Pillow*. Kompresi *JPEG* digunakan karena mampu mengurangi ukuran *file* dengan mengatur parameter kualitas. Oleh sebab itu, pembahasan mengenai kebutuhan kompresi menjadi penghubung antara konsep gambar digital dan analisis algoritma *JPEG* pada subbab berikutnya.

2.4 Algoritma Kompresi JPEG

Algoritma kompresi *JPEG* merupakan salah satu metode kompresi gambar digital yang banyak digunakan untuk gambar fotografi atau gambar dengan variasi warna yang kompleks. *JPEG* bekerja dengan pendekatan *lossy compression*, yaitu mengurangi sebagian informasi visual untuk menghasilkan ukuran *file* yang lebih kecil. MDN Web Docs menjelaskan bahwa *JPEG* merupakan format kompresi *lossy* yang banyak digunakan untuk gambar diam, terutama foto, tetapi kurang ideal untuk gambar yang membutuhkan ketajaman tinggi seperti diagram atau grafik (MDN Web Docs, 2025a).

Secara teknis, standar *JPEG* dirancang untuk kompresi dan pengkodean gambar diam bertipe *continuous-tone*, baik gambar *grayscale* maupun berwarna (ITU-T, 1993). Pada proses kompresinya, *JPEG* tidak langsung menyimpan seluruh

informasi gambar sebagaimana bentuk aslinya, tetapi mengubah gambar melalui beberapa tahapan agar data visual dapat direpresentasikan secara lebih efisien. Tahapan tersebut meliputi konversi ruang warna, pemrosesan blok gambar, transformasi menggunakan *Discrete Cosine Transform* atau *DCT*, *quantization*, dan *entropy coding*.

Wallace menjelaskan bahwa proses dasar pada *JPEG* berbasis *DCT* mencakup pembagian gambar ke dalam blok-blok kecil, transformasi frekuensi, kuantisasi koefisien, dan pengkodean akhir untuk mengurangi ukuran data (Gregory K., 1992). Pada laporan ini, pembahasan algoritma *JPEG* diarahkan pada empat bagian utama, yaitu konversi ruang warna *RGB* ke *YCbCr*, *Discrete Cosine Transform*, *quantization*, dan *entropy coding*. Keempat tahapan tersebut menjadi dasar untuk memahami mengapa pengaturan kualitas pada *JPEG* dapat memengaruhi ukuran *file* dan kualitas visual gambar.

2.4.1 Konversi Ruang Warna *RGB* ke *YCbCr*

Tahap awal dalam kompresi *JPEG* pada gambar berwarna adalah konversi ruang warna. Gambar digital umumnya direpresentasikan dalam ruang warna *RGB*, yaitu ruang warna yang terdiri dari komponen merah, hijau, dan biru. Representasi *RGB* umum digunakan pada perangkat tampilan karena warna gambar dapat dibentuk melalui kombinasi ketiga komponen tersebut. Namun, untuk kebutuhan kompresi, representasi *RGB* tidak selalu menjadi bentuk yang paling efisien.

Pada kompresi *JPEG*, ruang warna *RGB* dapat dikonversi ke *YCbCr*. Komponen *Y* merepresentasikan luminansi atau tingkat kecerahan gambar, sedangkan *Cb* dan *Cr* merepresentasikan informasi krominansi atau perbedaan warna. Pemisahan ini dilakukan karena mata manusia cenderung lebih peka terhadap perubahan kecerahan dibandingkan perubahan warna. Oleh karena itu, informasi warna dapat dikurangi dengan lebih agresif dibandingkan informasi kecerahan tanpa selalu menghasilkan penurunan kualitas visual yang terlalu mencolok.

Konversi ke *YCbCr* membantu proses kompresi karena sistem dapat memperlakukan informasi luminansi dan krominansi secara berbeda. Pada gambar

fotografi, detail kecerahan biasanya lebih penting untuk mempertahankan bentuk dan struktur objek, sedangkan detail warna tertentu masih dapat dikurangi. Shawahna, Haque, dan Amin menjelaskan bahwa salah satu tahapan kompresi *JPEG* adalah *color space conversion*, sebelum proses seperti *down sampling*, *DCT*, *quantization*, dan *entropy encoding* dilakukan (Shawahna et al., 2019).

Pada konteks laporan ini, konversi ruang warna penting untuk menjelaskan mengapa kompresi *JPEG* dapat mengurangi ukuran *file* tanpa langsung membuat gambar terlihat rusak secara drastis. Dengan memisahkan kecerahan dan warna, algoritma *JPEG* dapat memanfaatkan karakteristik persepsi manusia terhadap gambar. Tahap ini menjadi dasar sebelum gambar diproses lebih lanjut menggunakan transformasi frekuensi pada tahap *DCT*.

2.4.2 Discrete Cosine Transform (DCT)

Setelah gambar dikonversi ke ruang warna yang lebih sesuai, tahap berikutnya adalah pemrosesan menggunakan *Discrete Cosine Transform* atau *DCT*. *DCT* merupakan teknik transformasi yang mengubah data gambar dari domain spasial ke domain frekuensi. Domain spasial menggambarkan nilai *pixel* berdasarkan posisi pada gambar, sedangkan domain frekuensi menggambarkan perubahan intensitas atau pola visual dalam bentuk komponen frekuensi.

Pada kompresi *JPEG*, gambar umumnya diproses dalam blok kecil berukuran 8×8 *pixel*. Setiap blok kemudian diubah menggunakan *DCT* untuk menghasilkan sekumpulan koefisien frekuensi. Koefisien frekuensi rendah biasanya menyimpan informasi utama yang lebih berpengaruh terhadap tampilan gambar, sedangkan koefisien frekuensi tinggi sering berkaitan dengan detail halus atau perubahan tajam. John menjelaskan bahwa *DCT* memiliki peran penting dalam kompresi *JPEG* karena digunakan untuk merepresentasikan informasi gambar ke dalam komponen frekuensi yang lebih mudah dikompresi (John, 2021).

Peran utama *DCT* bukan langsung memperkecil ukuran *file*, melainkan menyiapkan data agar tahap kompresi berikutnya dapat bekerja lebih efektif. Setelah data gambar berada dalam domain frekuensi, sistem dapat menentukan bagian mana

yang lebih penting dan bagian mana yang dapat dikurangi. Proses ini sangat berhubungan dengan cara kerja *JPEG* sebagai kompresi *lossy*, karena pengurangan informasi biasanya dilakukan terhadap komponen yang dianggap kurang berpengaruh secara visual.

Untuk kebutuhan analisis kompresi gambar, pemahaman terhadap *DCT* membantu menjelaskan mengapa kualitas gambar dapat berubah ketika nilai kompresi diatur. Semakin kuat proses pengurangan terhadap komponen frekuensi tertentu, semakin kecil ukuran *file* yang dapat dihasilkan, tetapi detail visual juga dapat berkurang. Oleh karena itu, *DCT* menjadi tahap penting yang menghubungkan representasi gambar digital dengan proses *quantization*.

2.4.3 Quantization

Quantization merupakan tahap penting dalam algoritma *JPEG* karena pada tahap inilah sebagian informasi gambar mulai dikurangi. Setelah blok gambar ditransformasikan menggunakan *DCT*, hasilnya berupa koefisien frekuensi. Koefisien tersebut kemudian dibagi dengan nilai tertentu pada tabel kuantisasi, lalu dibulatkan. Proses pembulatan ini menyebabkan sebagian detail hilang, terutama pada komponen frekuensi tinggi yang dianggap kurang berpengaruh terhadap persepsi visual.

Tahap *quantization* menjadi sumber utama sifat *lossy* pada kompresi *JPEG*. Wallace menjelaskan bahwa setelah *DCT*, koefisien hasil transformasi akan dikuantisasi, dan tahap ini merupakan bagian yang menyebabkan hilangnya informasi pada proses kompresi (Gregory K., 1992). Dengan kata lain, ukuran *file* dapat diperkecil karena tidak semua informasi frekuensi disimpan secara utuh. Namun, semakin besar pengurangan yang dilakukan, semakin besar pula kemungkinan penurunan kualitas gambar.

Pada praktiknya, tingkat *quantization* berhubungan dengan pengaturan kualitas pada penyimpanan *JPEG*. Nilai kualitas yang lebih tinggi cenderung mempertahankan lebih banyak informasi sehingga ukuran *file* lebih besar, sedangkan nilai kualitas yang lebih rendah menghasilkan ukuran *file* lebih kecil

dengan risiko penurunan kualitas visual. Tahap ini menjelaskan mengapa parameter *quality* pada *Python Pillow* dapat memengaruhi hasil kompresi gambar.

Pada konteks laporan ini, *quantization* menjadi konsep utama untuk memahami hubungan antara ukuran *file* dan kualitas gambar. Analisis kompresi *JPEG* tidak cukup hanya melihat hasil akhir berupa *file* yang lebih kecil, tetapi juga perlu memahami bahwa pengurangan ukuran tersebut terjadi karena sebagian informasi visual tidak lagi disimpan secara penuh. Oleh karena itu, tahap *quantization* menjadi dasar penting sebelum membahas *entropy coding* sebagai tahap pengkodean akhir.

2.4.4 Entropy Coding

Setelah proses *quantization*, data gambar masih perlu dikodekan agar dapat disimpan dalam bentuk yang lebih ringkas. Tahap ini disebut *entropy coding*. Berbeda dengan *quantization* yang dapat menghilangkan sebagian informasi, *entropy coding* bertujuan mengurangi redundansi pada data hasil kuantisasi tanpa menambah kehilangan informasi baru. Dengan demikian, tahap ini bekerja sebagai proses kompresi tambahan setelah informasi gambar disederhanakan.

Pada algoritma *JPEG*, *entropy coding* umumnya dilakukan dengan memanfaatkan pola kemunculan nilai pada koefisien hasil kuantisasi. Nilai yang sering muncul dapat direpresentasikan dengan kode yang lebih pendek, sedangkan nilai yang jarang muncul dapat menggunakan kode yang lebih panjang. Wallace menjelaskan bahwa tahap akhir pada pemrosesan *JPEG* berbasis *DCT* adalah *entropy coding*, yang melakukan kompresi tambahan secara *lossless* terhadap koefisien *DCT* yang telah dikuantisasi (Gregory K., 1992).

Salah satu teknik yang digunakan dalam proses *entropy coding* adalah *Huffman coding*. Teknik ini mengatur panjang kode berdasarkan frekuensi kemunculan data, sehingga simbol yang lebih sering muncul dapat direpresentasikan dengan kode yang lebih pendek. Pada standar *JPEG*, pengkodean entropi menjadi bagian dari proses penyusunan data akhir agar hasil kompresi dapat disimpan secara lebih efisien (ITU-T, 1993).

Pada laporan ini, *entropy coding* dipahami sebagai tahap akhir yang membantu memperkecil ukuran *file* setelah tahap *DCT* dan *quantization*. Jika *quantization* berperan dalam mengurangi informasi visual, maka *entropy coding* berperan dalam menyusun kembali data hasil kuantisasi agar lebih hemat ruang penyimpanan. Dengan memahami tahapan ini, alur kompresi *JPEG* dapat dilihat sebagai rangkaian proses yang saling berkaitan, mulai dari pengolahan warna, transformasi frekuensi, pengurangan informasi, hingga pengkodean akhir.

Secara ringkas, tahapan umum algoritma kompresi *JPEG* dapat dilihat pada tabel berikut.

Tabel 2.4 Tahapan Umum Algoritma Kompresi *JPEG*

Tahapan	Fungsi Utama	Dampak terhadap Kompresi
Konversi <i>RGB</i> ke <i>YCbCr</i>	Memisahkan informasi kecerahan dan warna.	Membantu kompresi memanfaatkan karakteristik persepsi manusia.
<i>Discrete Cosine Transform (DCT)</i>	Mengubah data dari domain spasial ke domain frekuensi.	Memisahkan informasi utama dan detail frekuensi tinggi.
<i>Quantization</i>	Mengurangi presisi koefisien frekuensi.	Menjadi tahap utama yang menyebabkan penurunan ukuran <i>file</i> dan potensi penurunan kualitas.
<i>Entropy Coding</i>	Mengkodekan data hasil kuantisasi secara lebih ringkas.	Mengurangi redundansi tanpa menambah kehilangan informasi baru.

Tabel tersebut menunjukkan bahwa kompresi *JPEG* tidak terjadi dalam satu proses tunggal, melainkan melalui beberapa tahapan yang saling mendukung. Konversi ruang warna dan *DCT* menyiapkan data agar lebih mudah dikompresi, *quantization* mengurangi informasi untuk memperkecil ukuran *file*, sedangkan *entropy coding*

menyusun hasil akhir agar lebih efisien. Tahapan ini menjadi dasar untuk memahami bagaimana kompresi *JPEG* dapat dianalisis menggunakan *Python Pillow* pada Bab 3 Rancangan dan Implementasi.

2.5 *Python Pillow*

Python Pillow merupakan pustaka pengolahan gambar pada bahasa pemrograman *Python* yang digunakan untuk membuka, memanipulasi, mengonversi, dan menyimpan gambar dalam berbagai format. *Pillow* dikenal sebagai pengembangan dari *Python Imaging Library* atau *PIL*, sehingga sering disebut sebagai *PIL Fork*. Dokumentasi resmi *Pillow* menjelaskan bahwa *Pillow* menyediakan dukungan format *file* yang luas, representasi internal gambar yang efisien, serta kemampuan pemrosesan gambar yang dapat digunakan sebagai dasar pengolahan citra (Pillow, 2026c).

Pada konteks laporan ini, *Python Pillow* digunakan karena mampu menangani proses dasar kompresi gambar, khususnya pada format *JPEG*. Pustaka ini memungkinkan gambar dibuka, diproses, kemudian disimpan kembali dengan pengaturan tertentu. Proses penyimpanan ulang gambar dalam format *JPEG* dapat melibatkan parameter seperti *quality*, *optimize*, dan pengaturan lain yang berhubungan dengan hasil kompresi (Pillow, 2026d).

Pembahasan mengenai *Python Pillow* menjadi penting karena laporan ini tidak hanya membahas algoritma *JPEG* secara teori, tetapi juga mengaitkannya dengan penerapan kompresi gambar menggunakan pustaka tersebut. Dengan memahami fungsi dasar *Pillow*, fitur kompresi gambar, dan parameter *quality*, proses analisis pada Bab 3 dapat diarahkan untuk melihat hubungan antara pengaturan kompresi, ukuran *file*, dan kualitas visual gambar hasil kompresi.

2.5.1 Sekilas tentang *Python Pillow*

Python Pillow adalah pustaka tambahan pada bahasa pemrograman *Python* yang berfungsi untuk pengolahan gambar. Pustaka ini merupakan kelanjutan dari *Python Imaging Library* atau *PIL* yang sebelumnya banyak digunakan untuk kebutuhan manipulasi gambar. Situs resmi *Pillow* menyebut *Pillow* sebagai *friendly PIL fork*,

yang menunjukkan bahwa pustaka ini dikembangkan untuk melanjutkan dan memperbarui kemampuan *PIL* (Pillow, 2026d).

Sebagai pustaka pengolahan gambar, *Pillow* menyediakan berbagai kemampuan dasar seperti membuka *file* gambar, membaca informasi gambar, mengubah ukuran, mengonversi format, melakukan pemrosesan sederhana, dan menyimpan gambar kembali ke format tertentu. PyPI menjelaskan bahwa pustaka ini memberikan dukungan format *file* yang luas dan kemampuan pemrosesan gambar yang cukup kuat untuk digunakan pada kebutuhan pengolahan citra (Pillow, 2026d).

Pada laporan ini, *Python Pillow* dipilih karena sesuai dengan kebutuhan analisis kompresi gambar berbasis *JPEG*. Penggunaan *Pillow* memungkinkan proses kompresi dilakukan secara sederhana tanpa harus membangun algoritma *JPEG* dari awal. Dengan demikian, pembahasan *Pillow* pada Bab 2 berfungsi sebagai landasan untuk memahami alat yang digunakan dalam proses kompresi gambar pada Bab 3.

2.5.2 Fitur *Pillow* untuk Kompresi Gambar

Fitur utama *Pillow* yang relevan dengan kompresi gambar adalah kemampuan untuk membuka gambar dari berbagai format dan menyimpannya kembali dengan format serta parameter tertentu. Melalui mekanisme penyimpanan gambar, pengguna dapat menentukan format keluaran seperti *JPEG* dan mengatur parameter yang memengaruhi hasil kompresi. Dokumentasi resmi *Pillow* menjelaskan bahwa penyimpanan *file JPEG* mendukung sejumlah opsi, termasuk *quality*, *optimize*, *progressive*, dan *subsampling* (Pillow, 2026a).

Dalam proses kompresi gambar, fitur penyimpanan ulang sangat penting karena ukuran *file* akhir dapat berubah sesuai format dan parameter yang digunakan. Gambar yang dibuka dengan *Pillow* dapat disimpan kembali sebagai *JPEG* sehingga proses kompresi dapat dilakukan pada tahap penyimpanan. Dokumentasi modul *Image* pada *Pillow* juga menjelaskan bahwa untuk data gambar terkompresi seperti *PNG* atau *JPEG*, proses penyimpanan dapat dilakukan melalui fungsi penyimpanan gambar (Pillow, 2026b).

Selain menyimpan ulang gambar, *Pillow* juga dapat digunakan untuk menyesuaikan gambar sebelum dikompresi, misalnya mengubah ukuran atau mengonversi mode warna jika diperlukan. Namun, fokus laporan ini bukan pada seluruh fitur pengolahan citra yang tersedia pada *Pillow*, melainkan pada fitur yang berhubungan langsung dengan kompresi gambar *JPEG*. Oleh karena itu, pembahasan diarahkan pada proses membuka gambar, menyimpan gambar dalam format *JPEG*, dan mengatur parameter yang memengaruhi hasil kompresi.

Pada konteks analisis kompresi, fitur-fitur tersebut membantu melihat bagaimana gambar yang sama dapat menghasilkan ukuran *file* berbeda ketika disimpan dengan pengaturan kualitas yang berbeda. Hal ini sejalan dengan tujuan laporan, yaitu menganalisis hubungan antara kompresi *JPEG*, ukuran *file*, dan kualitas visual gambar. Dengan demikian, *Pillow* berperan sebagai alat penerapan kompresi, sedangkan konsep *JPEG* menjadi dasar teorinya.

2.5.3 Parameter *Quality* pada *JPEG*

Parameter *quality* merupakan salah satu pengaturan penting ketika menyimpan gambar dalam format *JPEG* menggunakan *Pillow*. Parameter ini digunakan untuk menentukan tingkat kualitas gambar hasil penyimpanan. Dokumentasi resmi *Pillow* menjelaskan bahwa nilai *quality* pada *JPEG* berada pada skala 0 sampai 95, dengan nilai bawaan 75, sedangkan nilai di atas 95 sebaiknya dihindari karena dapat menghasilkan *file* besar dengan peningkatan kualitas yang sangat kecil.

Secara konsep, nilai *quality* berhubungan dengan tingkat kompresi yang diterapkan pada gambar. Nilai *quality* yang lebih tinggi cenderung mempertahankan lebih banyak informasi visual, sehingga kualitas gambar lebih baik tetapi ukuran *file* lebih besar. Sebaliknya, nilai *quality* yang lebih rendah menghasilkan ukuran *file* lebih kecil, tetapi dapat menurunkan kualitas visual gambar. Pada format *JPEG*, hubungan ini berkaitan dengan sifat *lossy compression*, karena sebagian informasi gambar dapat dikurangi selama proses kompresi.

Dokumentasi *Pillow* juga menjelaskan bahwa nilai *quality* dapat menggunakan opsi *keep* untuk mempertahankan tingkat kualitas, *subsampling*, dan *quantization tables*

dari gambar *JPEG* asli. Hal ini menunjukkan bahwa parameter *quality* tidak hanya berhubungan dengan tampilan gambar, tetapi juga berkaitan dengan cara *JPEG* menyimpan dan mengatur informasi kompresi. Namun, pada laporan ini, pembahasan parameter *quality* difokuskan pada pengaruhnya terhadap ukuran *file* dan kualitas visual hasil kompresi.

Dalam analisis kompresi gambar, parameter *quality* menjadi variabel penting karena dapat digunakan untuk membandingkan hasil kompresi pada beberapa tingkat kualitas. Melalui pengaturan ini, laporan dapat menunjukkan bagaimana perubahan nilai *quality* memengaruhi hasil akhir gambar *JPEG*. Oleh karena itu, pemahaman terhadap parameter *quality* diperlukan sebelum masuk ke Bab 3 Rancangan dan Implementasi terutama pada bagian yang membandingkan ukuran *file* dan kualitas visual gambar hasil kompresi.

3. RANCANGAN DAN IMPLEMENTASI

3.1 Gambaran Umum Program

Program kompresi gambar yang dikembangkan merupakan implementasi kompresi gambar berformat *JPEG* menggunakan bahasa pemrograman *Python* dan pustaka *Pillow*. Program ini disusun dalam satu berkas *notebook* bernama `notebook/kompresi_gambar_pillow.ipynb` yang memuat bagian penjelasan, proses kompresi, perhitungan metrik, visualisasi, serta penyimpanan hasil pengujian. Berdasarkan laporan project, *notebook* tersebut terdiri dari 8 *cell markdown* dan 8 *cell code* yang telah dieksekusi secara lengkap, serta menghasilkan gambar kompresi dan *file* hasil pengujian dalam format *CSV* dan *Markdown*.

Secara umum, program ini digunakan untuk menguji proses kompresi gambar *JPEG* dengan bantuan *Python Pillow*. Pengujian diarahkan untuk membandingkan ukuran *file* gambar asli dan gambar hasil kompresi, menghitung metrik kualitas menggunakan *MSE* dan *PSNR*, serta menampilkan perbandingan visual antara gambar sebelum dan sesudah kompresi. Dengan demikian, program tidak hanya menghasilkan gambar terkompresi, tetapi juga menyediakan data pendukung untuk menilai dampak kompresi terhadap ukuran *file* dan kualitas gambar.

Pembahasan pada subbab ini difokuskan pada gambaran umum program, meliputi tujuan program, bahasa pemrograman dan *library* yang digunakan, serta jenis kompresi yang diterapkan. Detail mengenai struktur folder, parameter kompresi, proses implementasi, dan hasil pengujian tidak dibahas secara mendalam pada bagian ini karena akan dijelaskan pada subbab berikutnya.

3.1.1 Tujuan Program

Program ini dibuat untuk melakukan kompresi gambar menggunakan format *JPEG* dengan bantuan pustaka *Python Pillow*. Tujuan utama program adalah memperkecil ukuran *file* gambar melalui proses kompresi, kemudian membandingkan ukuran *file* asli dengan ukuran *file* hasil kompresi. Perbandingan tersebut menjadi dasar untuk

melihat seberapa besar perubahan ukuran *file* setelah gambar disimpan ulang dalam format *JPEG*.

Selain membandingkan ukuran *file*, program juga digunakan untuk menghitung metrik kualitas gambar hasil kompresi. Metrik yang digunakan adalah *Mean Squared Error* atau *MSE* dan *Peak Signal-to-Noise Ratio* atau *PSNR*. Kedua metrik tersebut digunakan untuk memberikan gambaran kuantitatif mengenai perbedaan antara gambar asli dan gambar hasil kompresi. Dengan adanya pengukuran tersebut, hasil kompresi tidak hanya dilihat dari sisi ukuran *file*, tetapi juga dari sisi perubahan kualitas gambar.

Program ini juga mendukung perbandingan visual antara gambar asli dan gambar hasil kompresi. Perbandingan visual diperlukan karena hasil kompresi gambar tidak selalu cukup dinilai melalui angka metrik saja. Pada laporan ini, tujuan program dibatasi pada analisis kompresi gambar *JPEG* menggunakan *Python Pillow*, bukan pada pembuatan algoritma *JPEG* dari awal. Proses seperti *DCT*, *quantization*, dan *entropy coding* dijalankan secara internal oleh *Pillow* sebagai bagian dari mekanisme penyimpanan gambar *JPEG*.

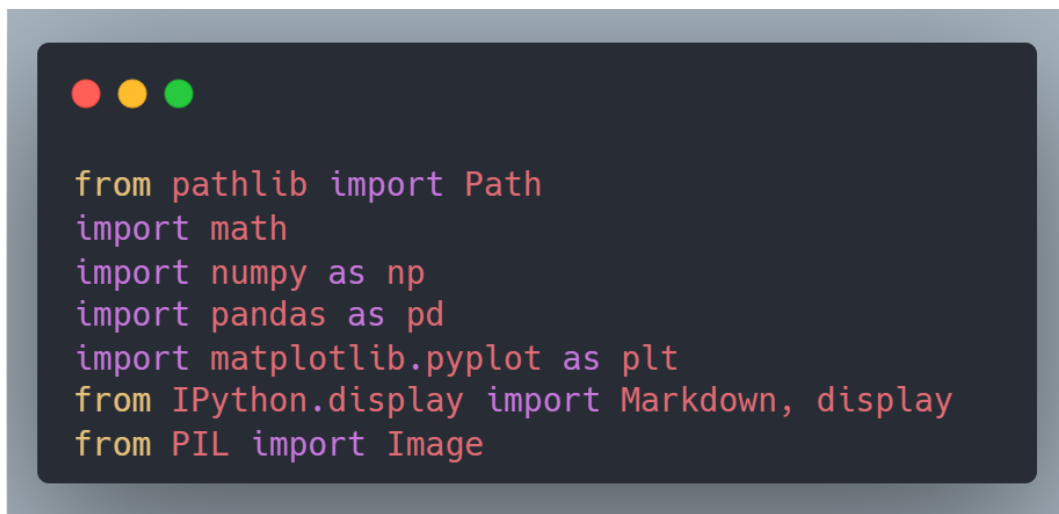
3.1.2 Bahasa Pemrograman dan *Library* yang Digunakan

Program ini ditulis menggunakan bahasa pemrograman *Python 3*. Pemilihan *Python* mendukung proses pengolahan gambar karena memiliki ekosistem pustaka yang memadai untuk manipulasi gambar, perhitungan numerik, pengolahan tabel, dan visualisasi. Seluruh proses pada program dijalankan melalui *Jupyter Notebook*, sehingga tahapan penjelasan, implementasi, pengujian, dan penampilan hasil dapat disusun secara runtut dalam satu dokumen kerja.

Beberapa *library* yang digunakan dalam program meliputi *Pillow*, *NumPy*, *Pandas*, *Matplotlib*, *pathlib*, *math*, dan *tabulate*. *Pillow* digunakan untuk membuka, memproses, dan menyimpan gambar dalam format *JPEG*. *NumPy* digunakan untuk mendukung perhitungan numerik, termasuk perhitungan berbasis nilai *pixel*. *Pandas* digunakan untuk menyusun hasil pengujian dalam bentuk tabel, sedangkan *Matplotlib* digunakan untuk menampilkan perbandingan visual gambar. Sementara

itu, *pathlib* membantu pengelolaan lokasi *file* dan folder, *math* digunakan untuk operasi matematis, dan *tabulate* digunakan untuk menyusun keluaran tabel dalam format *Markdown*.

Cuplikan berikut menunjukkan daftar *library* yang digunakan pada bagian awal program.

A screenshot of a code editor window with a dark background and light-colored text. The window has three colored window control buttons (red, yellow, green) in the top-left corner. The code displayed is a list of Python import statements for various libraries used in the program.

```
from pathlib import Path
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Markdown, display
from PIL import Image
```

Gambar 3.1 Daftar *Library* yang Digunakan

Cuplikan tersebut menunjukkan bahwa program menggunakan kombinasi pustaka untuk pengolahan gambar, perhitungan numerik, pengelolaan data, dan visualisasi. Penggunaan *from PIL import Image* menjadi bagian utama karena proses kompresi gambar *JPEG* dilakukan melalui pustaka *Pillow*. Sementara itu, pustaka lain berperan sebagai pendukung agar hasil kompresi dapat dihitung, disusun, dan ditampilkan secara lebih terstruktur.

3.1.3 Jenis Kompresi yang Digunakan

Jenis kompresi yang digunakan pada program ini adalah kompresi *lossy* melalui format *JPEG*. Kompresi *lossy* bekerja dengan mengurangi sebagian informasi pada gambar untuk menghasilkan ukuran *file* yang lebih kecil. Pada konteks gambar digital, pendekatan ini masih dapat diterima selama penurunan kualitas visual tidak terlalu mengganggu tampilan gambar.

Format *JPEG* dipilih karena sesuai dengan fokus laporan, yaitu analisis kompresi gambar menggunakan algoritma *JPEG* dengan *Python Pillow*. Secara umum, kompresi *JPEG* melibatkan beberapa tahapan, seperti konversi ruang warna *RGB* ke *YCbCr*, pembagian gambar ke dalam blok 8×8 , proses *Discrete Cosine Transform* atau *DCT*, *quantization*, dan *entropy coding*. Namun, pada program ini, tahapan tersebut tidak diimplementasikan secara manual dari awal. Proses kompresi *JPEG* dijalankan melalui fitur internal *Pillow* ketika gambar disimpan dalam format *JPEG*.

Penggunaan kompresi *lossy* pada program ini berkaitan langsung dengan tujuan analisis, yaitu melihat perubahan ukuran *file* dan kualitas gambar setelah proses kompresi. Karena *JPEG* mengurangi sebagian informasi gambar, hasil kompresi perlu dievaluasi melalui ukuran *file*, metrik *MSE*, metrik *PSNR*, serta perbandingan visual. Dengan demikian, jenis kompresi yang digunakan sudah sesuai dengan arah laporan, yaitu menganalisis hubungan antara kompresi *JPEG*, ukuran *file*, dan kualitas gambar hasil kompresi.

3.2 Alur Kerja Program

Alur kerja program kompresi gambar disusun secara berurutan mulai dari penerimaan gambar, proses konversi dan kompresi, penyimpanan hasil, sampai perhitungan hasil pengujian. Seluruh proses dijalankan melalui *Jupyter Notebook* sehingga tahapan kerja dapat dilakukan dari atas ke bawah dalam satu alur eksekusi. Berdasarkan laporan project, program memindai folder `data/input/original/`, memproses gambar yang ditemukan, menyimpan hasil kompresi ke folder `data/output/compressed/`, serta mengeksport hasil pengujian ke folder `data/results/` dalam format *CSV* dan *Markdown*.

Secara umum, alur program dimulai ketika gambar uji ditempatkan pada folder *input*. Program kemudian membaca *file* gambar berdasarkan ekstensi yang didukung, membuka gambar menggunakan *Pillow*, mengonversinya ke mode *RGB*, lalu menyimpan ulang gambar dalam format *JPEG*. Setelah proses kompresi selesai, program menghitung ukuran *file* asli dan hasil kompresi, kemudian menghitung metrik pengujian berupa rasio kompresi, *MSE*, dan *PSNR*.

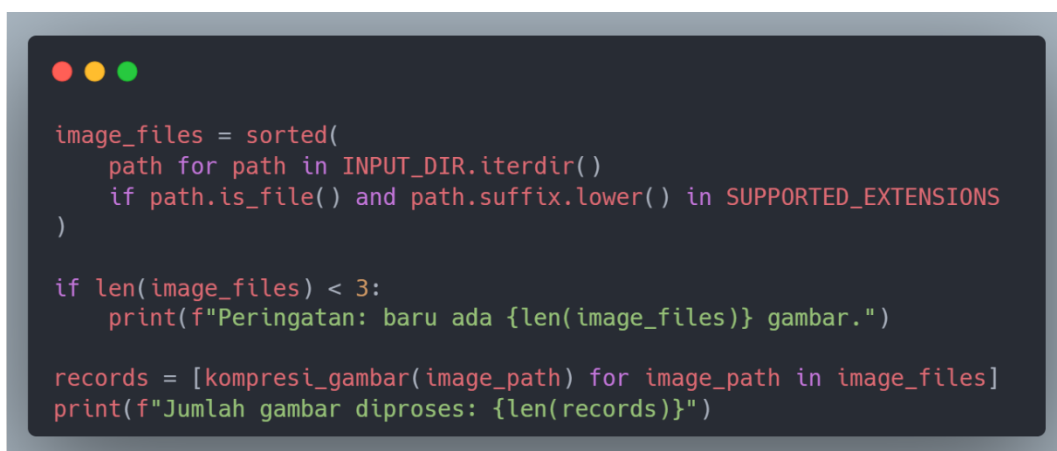
Alur kerja tersebut menunjukkan bahwa program tidak hanya melakukan penyimpanan ulang gambar, tetapi juga menghasilkan data evaluasi yang dapat digunakan untuk pembahasan. Dengan demikian, proses kompresi gambar pada laporan ini dapat ditelusuri dari gambar masukan sampai *file* hasil pengujian. Urutan ini menjadi dasar untuk memahami hasil analisis pada subbab berikutnya, terutama pada bagian perhitungan metrik dan pembahasan hasil kompresi.

3.2.1 Input Gambar

Tahap pertama pada program adalah menyiapkan gambar uji sebagai masukan. Gambar ditempatkan pada folder `data/input/original/` sebelum *notebook* dijalankan. Program kemudian memindai folder tersebut untuk mencari *file* gambar dengan ekstensi yang didukung, yaitu *.jpg*, *.jpeg*, dan *.png*. Tahap ini memastikan bahwa hanya *file* gambar yang sesuai yang akan diproses lebih lanjut.

Pemindaian *file* dilakukan secara otomatis menggunakan *path* folder masukan. *File* yang ditemukan kemudian difilter berdasarkan ekstensi, diurutkan secara alfabetis, dan diproses satu per satu. Pada tahap ini, program juga memeriksa jumlah gambar yang ditemukan. Jika gambar yang tersedia kurang dari jumlah minimum yang diharapkan, program menampilkan peringatan.

Cuplikan berikut menunjukkan bagian program yang digunakan untuk memindai gambar masukan dan menjalankan proses kompresi untuk setiap *file* yang ditemukan.

A screenshot of a code editor with a dark background and light-colored text. The code is written in Python and is used to scan a directory for image files. It defines a list of supported extensions, iterates over the files in a directory, filters them by extension, and then processes them. A warning message is printed if the number of files found is less than 3. The code is as follows:

```
image_files = sorted(
    path for path in INPUT_DIR.iterdir()
    if path.is_file() and path.suffix.lower() in SUPPORTED_EXTENSIONS
)

if len(image_files) < 3:
    print(f"Peringatan: baru ada {len(image_files)} gambar.")

records = [kompresi_gambar(image_path) for image_path in image_files]
print(f"Jumlah gambar diproses: {len(records)}")
```

Gambar 3.2 Pemindaian Gambar Masukan

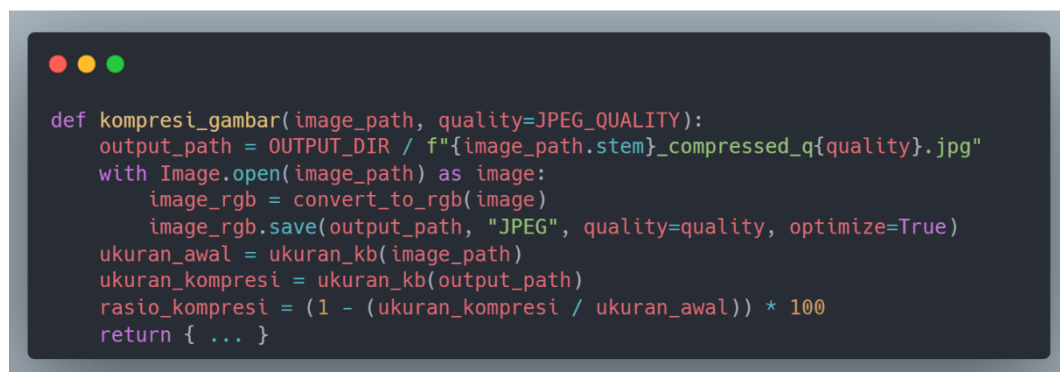
Cuplikan tersebut memperlihatkan bahwa program bekerja dengan cara membaca isi folder masukan, menyaring *file* berdasarkan ekstensi yang didukung, lalu memproses seluruh gambar yang ditemukan melalui fungsi `kompresi_gambar()`. Hasil eksekusi menunjukkan bahwa terdapat 3 gambar yang diproses. Dengan demikian, tahap input pada program sudah berjalan sesuai kebutuhan pengujian kompresi gambar.

3.2.2 Proses Konversi dan Kompresi

Setelah gambar masukan ditemukan, tahap berikutnya adalah proses konversi dan kompresi. Setiap gambar dibuka menggunakan *Pillow* melalui fungsi `Image.open()`. Gambar kemudian dikonversi ke mode *RGB* sebelum disimpan ulang sebagai *JPEG*. Konversi ke *RGB* diperlukan karena format *JPEG* tidak mendukung kanal transparansi, sehingga gambar yang memiliki transparansi perlu disesuaikan terlebih dahulu sebelum dikompresi.

Proses kompresi dilakukan dengan menyimpan ulang gambar ke format *JPEG*. Pada tahap ini, *Pillow* menjalankan mekanisme kompresi *JPEG* secara internal. Program tidak mengimplementasikan sendiri tahapan algoritma seperti *DCT*, *quantization*, atau *Huffman coding*, melainkan memanfaatkan fungsi penyimpanan gambar yang disediakan oleh *Pillow*. Dengan pendekatan ini, program dapat difokuskan pada analisis hasil kompresi, bukan pada pembuatan algoritma kompresi dari awal.

Bagian inti proses konversi dan kompresi ditunjukkan pada cuplikan berikut.

A screenshot of a code editor window with a dark background and light-colored text. The code defines a function named `kompresi_gambar` that takes `image_path` and `quality` as arguments. It calculates the output path, opens the image with `Image.open`, converts it to RGB, and saves it as a JPEG. It also calculates the original and compressed file sizes to determine the compression ratio.

```
def kompresi_gambar(image_path, quality=JPEG_QUALITY):
    output_path = OUTPUT_DIR / f"{image_path.stem}_compressed_q{quality}.jpg"
    with Image.open(image_path) as image:
        image_rgb = convert_to_rgb(image)
        image_rgb.save(output_path, "JPEG", quality=quality, optimize=True)
    ukuran_awal = ukuran_kb(image_path)
    ukuran_kompresi = ukuran_kb(output_path)
    rasio_kompresi = (1 - (ukuran_kompresi / ukuran_awal)) * 100
    return { ... }
```

Gambar 3.3 Proses Konversi dan Kompresi Gambar

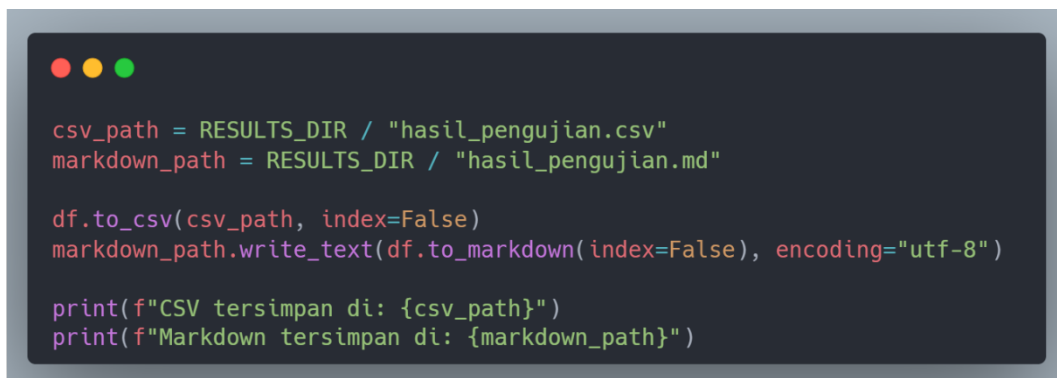
Cuplikan tersebut menunjukkan alur utama program, yaitu membuka gambar, mengonversi gambar ke mode *RGB*, menyimpan hasilnya sebagai *JPEG*, kemudian menghitung ukuran *file* asli dan ukuran *file* hasil kompresi. Perhitungan rasio kompresi juga dimulai pada fungsi yang sama, tetapi pembahasan angka dan interpretasi hasilnya tidak dijelaskan pada subbab ini agar tidak mendahului bagian hasil pengujian.

3.2.3 Penyimpanan Hasil Kompresi

Hasil kompresi gambar disimpan pada folder `data/output/compressed/`. Setiap gambar hasil kompresi disimpan dalam format *JPEG* dengan pola penamaan yang konsisten, yaitu menggunakan nama dasar *file* asli yang ditambahkan keterangan hasil kompresi. Pola penamaan ini membantu membedakan *file* asli dan *file* hasil kompresi, serta memudahkan proses pelacakan hasil pada tahap analisis.

Selain menyimpan gambar hasil kompresi, program juga menyimpan data hasil pengujian ke dalam folder `data/results/`. Data hasil pengujian dikumpulkan dalam bentuk *DataFrame* menggunakan *Pandas*, kemudian diekspor menjadi dua format, yaitu *CSV* dan *Markdown*. Format *CSV* digunakan untuk menyimpan data dalam bentuk tabel yang mudah dibaca kembali, sedangkan format *Markdown* membantu penyajian tabel hasil dalam bentuk teks yang lebih mudah ditempelkan ke laporan.

Cuplikan berikut menunjukkan proses penyimpanan hasil pengujian ke dalam *file CSV* dan *Markdown*.



```
csv_path = RESULTS_DIR / "hasil_pengujian.csv"
markdown_path = RESULTS_DIR / "hasil_pengujian.md"

df.to_csv(csv_path, index=False)
markdown_path.write_text(df.to_markdown(index=False), encoding="utf-8")

print(f"CSV tersimpan di: {csv_path}")
print(f"Markdown tersimpan di: {markdown_path}")
```

Gambar 3.4 Penyimpanan Hasil Pengujian

Cuplikan tersebut memperlihatkan bahwa hasil pengujian tidak hanya ditampilkan di dalam *notebook*, tetapi juga disimpan sebagai *file* terpisah. Penyimpanan ke format *CSV* dan *Markdown* membuat hasil pengujian lebih mudah digunakan kembali untuk pembuatan tabel, pemeriksaan data, dan penyusunan pembahasan pada bagian berikutnya.

3.2.4 Perhitungan Hasil Pengujian

Tahap terakhir dalam alur kerja program adalah perhitungan hasil pengujian. Setelah gambar berhasil dikompresi, program menghitung ukuran *file* asli dan ukuran *file* hasil kompresi. Ukuran *file* tersebut kemudian digunakan untuk menghitung rasio kompresi, yaitu persentase pengurangan ukuran *file* setelah proses kompresi dilakukan. Rasio ini menjadi metrik awal untuk melihat efektivitas kompresi dari sisi ukuran *file*.

Selain rasio kompresi, program juga menghitung *MSE* dan *PSNR*. *MSE* digunakan untuk mengukur rata-rata perbedaan nilai *pixel* antara gambar asli dan gambar hasil kompresi, sedangkan *PSNR* digunakan untuk memberikan gambaran kualitas gambar berdasarkan galat kompresi. Perhitungan kedua metrik tersebut dilakukan dengan membandingkan data *pixel* dari gambar asli dan gambar hasil kompresi yang telah diproses.

Hasil perhitungan kemudian dikumpulkan ke dalam struktur data dan disusun menjadi tabel menggunakan *Pandas DataFrame*. Data yang dihasilkan mencakup ukuran *file*, rasio kompresi, *MSE*, dan *PSNR*, lalu disimpan ke *file* hasil pengujian. Pada subbab ini, pembahasan dibatasi pada alur perhitungan. Nilai angka dan interpretasi dari metrik tersebut dijelaskan pada subbab hasil pengujian agar pembahasan tetap runtut.

Secara ringkas, alur kerja program dapat dijelaskan sebagai berikut.

1. Gambar uji ditempatkan pada *folder data/input/original/*.
2. Program memindai *folder* masukan dan memfilter *file* berdasarkan ekstensi gambar yang didukung.

3. Setiap gambar dibuka menggunakan *Pillow*.
4. Gambar dikonversi ke mode *RGB*.
5. Gambar disimpan ulang sebagai format *JPEG*.
6. Ukuran *file* asli dan hasil kompresi dihitung.
7. Rasio kompresi, *MSE*, dan *PSNR* dihitung.
8. Hasil kompresi disimpan ke *folder data/output/compressed/*.
9. Hasil pengujian disimpan ke *folder data/results/* dalam format *CSV* dan *Markdown*.

Urutan tersebut menunjukkan bahwa program memiliki alur kerja yang lengkap dari input sampai penyimpanan hasil. Dengan alur ini, hasil kompresi gambar dapat diperiksa melalui *file* gambar keluaran, sedangkan hasil pengujian kuantitatif dapat diperiksa melalui *file* tabel yang telah diekspor.

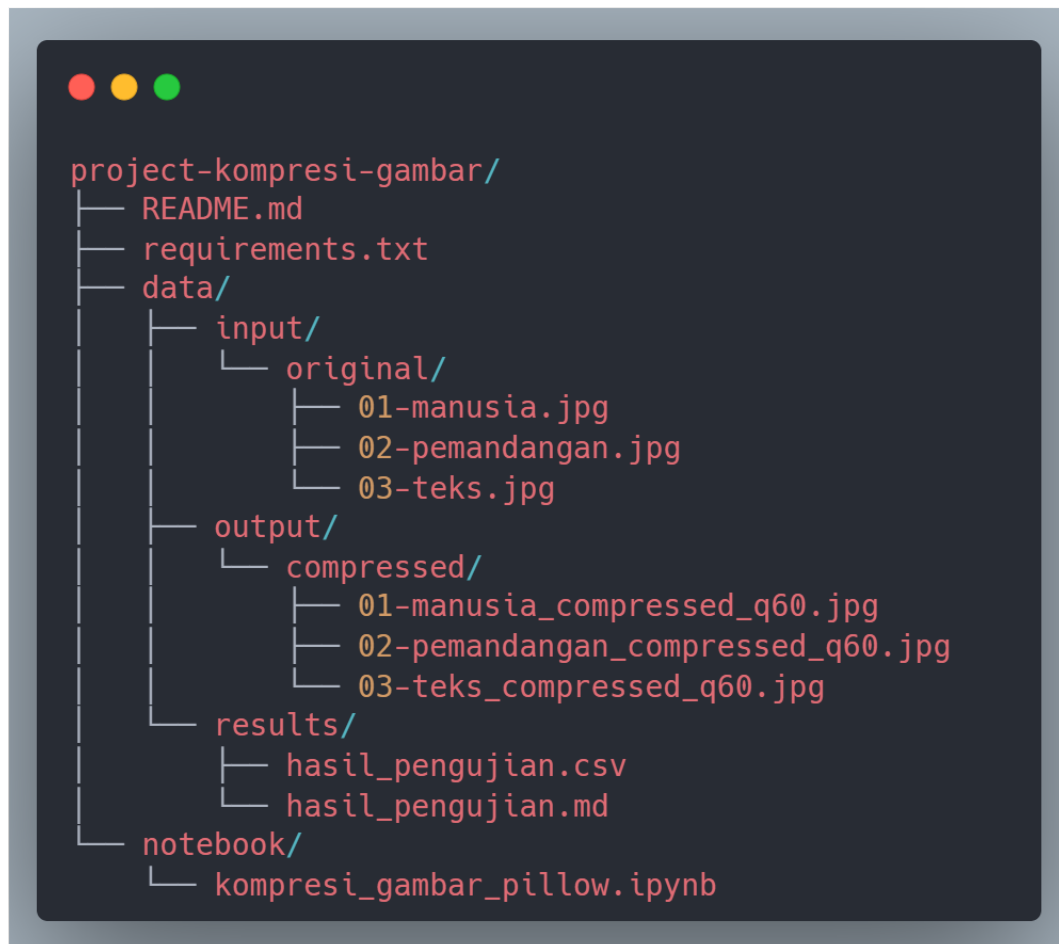
3.3 Struktur Folder Project

Struktur *folder project* disusun untuk memisahkan antara *file* program, gambar *input*, gambar *output* hasil kompresi, dan *file* hasil pengujian. Pemisahan ini membantu pengelolaan data menjadi lebih jelas karena setiap jenis *file* memiliki lokasi penyimpanan yang berbeda. Berdasarkan laporan *project*, struktur utama terdiri dari *folder data/* untuk menyimpan gambar dan hasil pengujian, serta *folder notebook/* untuk menyimpan *file* utama *Jupyter Notebook*.

Folder data/ memiliki tiga bagian utama, yaitu *data/input/original/* sebagai lokasi gambar asli, *data/output/compressed/* sebagai lokasi gambar hasil kompresi, dan *data/results/* sebagai lokasi *file* hasil pengujian. Sementara itu, *folder notebook/* berisi *file* program utama bernama *kompresi_gambar_pillow.ipynb*. *File* pendukung lain seperti *README.md* dan *requirements.txt* diletakkan pada bagian utama *project* untuk mendukung dokumentasi dan kebutuhan pustaka.

Struktur tersebut menunjukkan bahwa *project* tidak hanya berisi kode program, tetapi juga mencakup data masukan, data keluaran, serta hasil pengujian yang dapat

diperiksa kembali. Dengan organisasi *folder* seperti ini, proses analisis kompresi gambar dapat dilakukan secara lebih terarah karena gambar asli, gambar hasil kompresi, dan *file* hasil pengujian tidak bercampur dalam satu lokasi.



Gambar 3.5 Struktur *Folder Project*

Struktur tersebut memperlihatkan hubungan antara gambar asli, gambar hasil kompresi, dan *file* hasil pengujian. *Folder input* berperan sebagai sumber data, *folder output* menjadi tempat penyimpanan hasil kompresi, sedangkan *folder results* menyimpan data pengujian dalam bentuk tabel. Dengan demikian, setiap tahap pada program memiliki lokasi penyimpanan yang jelas.

3.3.1 *Folder Input Gambar*

Folder data/input/original/ digunakan sebagai tempat penyimpanan gambar asli sebelum diproses oleh program. Pada *folder* ini terdapat tiga gambar uji berformat

JPEG, yaitu *01-manusia.jpg*, *02-pemandangan.jpg*, dan *03-teks.jpg*. Ketiga gambar tersebut menjadi data *input* utama dalam proses kompresi menggunakan *Python Pillow*.

Gambar yang berada pada *folder input* memiliki resolusi tinggi dan ukuran *file* yang berbeda. Gambar *01-manusia.jpg* berukuran 4.274,61 KB dengan dimensi 5.231×3.487 *pixel*, gambar *02-pemandangan.jpg* berukuran 3.903,40 KB dengan dimensi 5.760×3.840 *pixel*, dan gambar *03-teks.jpg* berukuran 4.748,37 KB dengan dimensi 5.595×3.980 *pixel*. Total ukuran ketiga gambar *input* adalah 12.926,38 KB.

Pemilihan tiga gambar dengan karakteristik berbeda membantu proses pengujian karena setiap gambar dapat memberikan respons kompresi yang tidak selalu sama. Gambar manusia, pemandangan, dan teks memiliki variasi detail visual yang berbeda, sehingga dapat digunakan untuk melihat bagaimana kompresi *JPEG* bekerja pada beberapa jenis konten gambar. Pada bagian ini, pembahasan hanya menampilkan keberadaan dan karakteristik dasar gambar *input*, sedangkan analisis hasil kompresi dijelaskan pada subbab pengujian.

3.3.2 Folder Output Gambar Hasil Kompresi

Folder data/output/compressed/ digunakan untuk menyimpan gambar hasil kompresi. Setelah gambar asli diproses oleh program, hasilnya disimpan dalam format *JPEG* dengan pola penamaan yang konsisten. Pola tersebut menggunakan nama dasar *file* asli, kemudian ditambahkan keterangan *_compressed_q60.jpg* sebagai penanda bahwa *file* tersebut merupakan hasil kompresi.

Pada *folder* ini terdapat tiga gambar hasil kompresi, yaitu *01-manusia_compressed_q60.jpg*, *02-pemandangan_compressed_q60.jpg*, dan *03-teks_compressed_q60.jpg*. Ukuran masing-masing *file* hasil kompresi adalah 1.863,69 KB, 2.195,22 KB, dan 2.666,65 KB. Total ukuran *file output* hasil kompresi adalah 6.725,56 KB, lebih kecil dibandingkan total ukuran gambar asli pada *folder input*.

Meskipun ukuran *file* hasil kompresi lebih kecil, dimensi gambar tetap sama dengan gambar asli. Hal ini menunjukkan bahwa proses yang dilakukan bukan perubahan resolusi atau *resizing*, melainkan kompresi gambar dalam format *JPEG*. Dengan demikian, *folder output* berfungsi sebagai bukti bahwa program menghasilkan *file* gambar baru dari proses kompresi, tanpa mengubah ukuran dimensi gambar.

3.3.3 Folder Hasil Pengujian

Folder data/results/ digunakan untuk menyimpan *file* hasil pengujian kompresi. *File* hasil pengujian disimpan dalam dua format, yaitu *hasil_pengujian.csv* dan *hasil_pengujian.md*. Kedua *file* tersebut berisi data hasil pengujian yang sama, tetapi disajikan dalam format berbeda sesuai kebutuhan penggunaan.

File hasil_pengujian.csv digunakan untuk menyimpan hasil pengujian dalam format *CSV* agar mudah dibaca kembali menggunakan perangkat pengolah data atau pustaka seperti *Pandas*. Sementara itu, *file hasil_pengujian.md* digunakan untuk menyimpan hasil dalam format *Markdown*, sehingga lebih mudah digunakan sebagai bahan penyajian tabel dalam laporan. Ukuran *file hasil_pengujian.csv* adalah 0,51 KB, sedangkan *file hasil_pengujian.md* berukuran 0,96 KB.

Isi hasil pengujian pada *folder* ini mencakup informasi yang berkaitan dengan proses kompresi, seperti nama *file*, ukuran *file*, rasio kompresi, *MSE*, dan *PSNR*. Namun, bagian ini tidak membahas angka metrik secara rinci karena fokus subbab 3.3 adalah struktur penyimpanan *file*. Pembahasan nilai metrik dan interpretasi hasil akan dijelaskan pada subbab hasil pengujian.

3.3.4 File Notebook dan File Pendukung

File utama program berada pada *folder notebook/* dengan nama *kompresi_gambar_pillow.ipynb*. *File* tersebut merupakan *Jupyter Notebook* yang memuat penjelasan, kode program, proses kompresi, perhitungan metrik, visualisasi, dan penyimpanan hasil. Dengan menggunakan format *notebook*, proses pengembangan dan pengujian dapat disusun secara bertahap dalam satu *file* kerja.

Selain *file notebook* utama, terdapat *folder .ipynb_checkpoints/* yang merupakan *folder* otomatis dari *Jupyter*. *Folder* tersebut berisi *checkpoint* dari *file notebook*

dan bukan bagian utama dari hasil laporan. Keberadaan *folder* ini bersifat pendukung teknis, sehingga tidak digunakan sebagai sumber pembahasan utama.

Pada bagian utama *project*, terdapat dua *file* pendukung, yaitu *README.md* dan *requirements.txt*. *File README.md* berisi dokumentasi penggunaan dan struktur *project*, sedangkan *requirements.txt* berisi daftar pustaka yang dibutuhkan untuk menjalankan program. Dengan adanya *file* pendukung tersebut, *project* menjadi lebih mudah dipahami dan dijalankan kembali.

Tabel 3.1 Daftar *File* Gambar dan Ukuran

Nama File	Lokasi	Ukuran
01-manusia.jpg	data/input/original/	4.274,61 KB
02-pemandangan.jpg	data/input/original/	3.903,40 KB
03-teks.jpg	data/input/original/	4.748,37 KB
01-manusia_compressed_q60.jpg	data/output/compressed/	1.863,69 KB
02-pemandangan_compressed_q60.jpg	data/output/compressed/	2.195,22 KB
03-teks_compressed_q60.jpg	data/output/compressed/	2.666,65 KB

Tabel tersebut menunjukkan bahwa setiap gambar *input* memiliki pasangan gambar hasil kompresi. Keberadaan pasangan *file* ini membantu proses perbandingan antara gambar asli dan gambar hasil kompresi pada tahap pengujian. Namun, tabel ini belum digunakan untuk menghitung rasio kompresi karena pembahasan rasio dan interpretasi metrik ditempatkan pada subbab khusus hasil pengujian.

3.4 Implementasi Program

Implementasi program kompresi gambar dibuat dalam satu *file Jupyter Notebook* bernama *notebook/kompresi_gambar_pillow.ipynb*. Bagian kode program terdiri dari beberapa *cell code* yang menjalankan proses secara berurutan, mulai dari *import library*, penentuan *path folder*, pendefinisian fungsi, proses *batch compression*, penampilan hasil, visualisasi, hingga *export* hasil pengujian. Berdasarkan laporan *project*, implementasi utama berada pada enam *cell code* yang

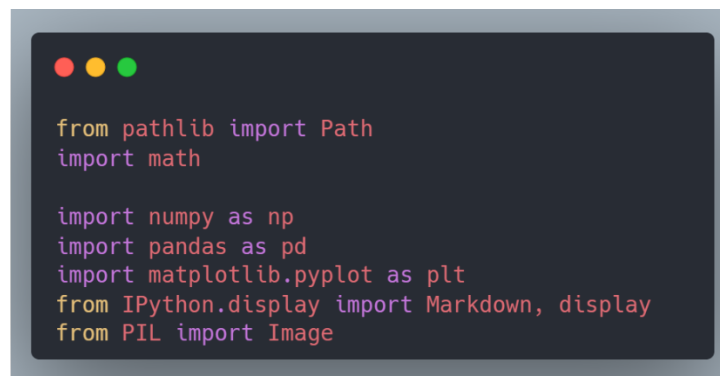
telah dieksekusi dan menghasilkan tiga gambar hasil kompresi beserta *file* hasil pengujian .

Secara umum, program bekerja dengan membuka gambar menggunakan *Pillow*, mengonversi gambar ke mode *RGB*, lalu menyimpannya kembali sebagai *JPEG* dengan parameter *quality* dan *optimize*. Setelah gambar berhasil dikompresi, program menghitung ukuran *file*, rasio kompresi, *MSE*, dan *PSNR*. Hasil perhitungan tersebut kemudian dikumpulkan dalam bentuk *dictionary*, diolah menjadi *DataFrame*, dan digunakan pada tahap penyajian hasil.

Implementasi ini tidak membuat algoritma *JPEG* dari awal. Proses seperti *DCT*, *quantization*, dan *entropy coding* dijalankan secara internal oleh *Pillow* ketika gambar disimpan dalam format *JPEG*. Oleh karena itu, bagian implementasi program lebih difokuskan pada bagaimana program menggunakan *library* yang tersedia untuk melakukan kompresi, menghitung hasil, dan menyimpan data pengujian secara terstruktur.

3.4.1 Import Library

Tahap awal implementasi dilakukan dengan mengimpor beberapa *library* yang dibutuhkan oleh program. *Library* utama yang digunakan adalah *Pillow* melalui modul *PIL.Image* untuk membuka, mengonversi, dan menyimpan gambar. Selain itu, program juga menggunakan *NumPy* untuk operasi numerik berbasis *array pixel*, *Pandas* untuk menyusun hasil pengujian dalam bentuk *DataFrame*, serta *Matplotlib* untuk menampilkan perbandingan visual gambar.



```
from pathlib import Path
import math

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Markdown, display
from PIL import Image
```

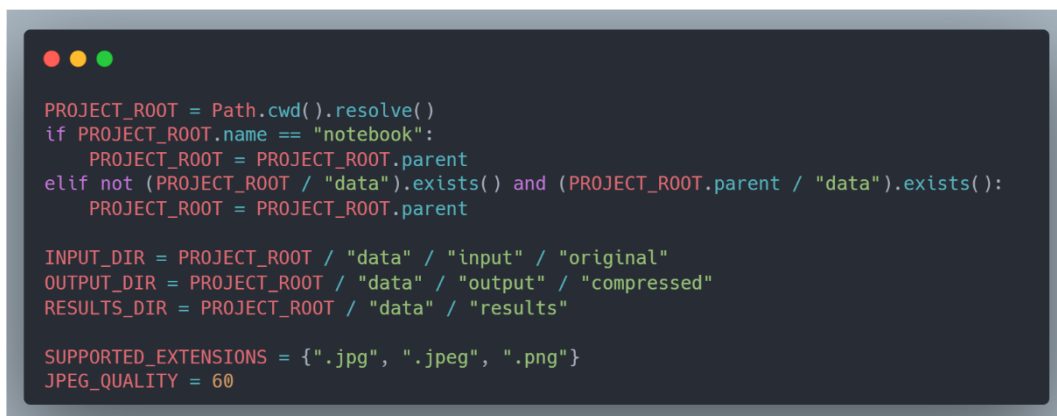
Gambar 3.6 Import Library

Cuplikan tersebut menunjukkan seluruh *library* yang digunakan pada program. *Pillow* menjadi *library* utama untuk proses kompresi gambar, sedangkan *NumPy*, *Pandas*, dan *Matplotlib* berperan sebagai pendukung untuk perhitungan metrik, penyusunan tabel, dan visualisasi. Pada bagian ini, versi setiap *library* tidak dituliskan karena tidak tercantum secara spesifik pada *requirements.txt*.

Library pathlib digunakan untuk mengelola *path folder* secara lebih rapi, sedangkan *math* digunakan untuk perhitungan matematis pada metrik *PSNR*. Program juga menggunakan *IPython.display* untuk menampilkan hasil dalam lingkungan *Jupyter Notebook*. Kombinasi beberapa *library* tersebut menunjukkan bahwa program tidak hanya berfokus pada kompresi gambar, tetapi juga mencakup proses perhitungan, penyajian data, dan visualisasi hasil.

3.4.2 Penentuan *Path Folder*

Setelah *library* diimpor, program menentukan beberapa *path folder* yang digunakan selama proses kompresi dan pengujian. *Path* utama yang didefinisikan meliputi *PROJECT_ROOT*, *INPUT_DIR*, *OUTPUT_DIR*, dan *RESULTS_DIR*. *INPUT_DIR* digunakan sebagai lokasi gambar asli, *OUTPUT_DIR* digunakan sebagai lokasi gambar hasil kompresi, sedangkan *RESULTS_DIR* digunakan sebagai lokasi penyimpanan hasil pengujian.



```
PROJECT_ROOT = Path.cwd().resolve()
if PROJECT_ROOT.name == "notebook":
    PROJECT_ROOT = PROJECT_ROOT.parent
elif not (PROJECT_ROOT / "data").exists() and (PROJECT_ROOT.parent / "data").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

INPUT_DIR = PROJECT_ROOT / "data" / "input" / "original"
OUTPUT_DIR = PROJECT_ROOT / "data" / "output" / "compressed"
RESULTS_DIR = PROJECT_ROOT / "data" / "results"

SUPPORTED_EXTENSIONS = {".jpg", ".jpeg", ".png"}
JPEG_QUALITY = 60
```

Gambar 3.7 Konfigurasi *Path Folder*

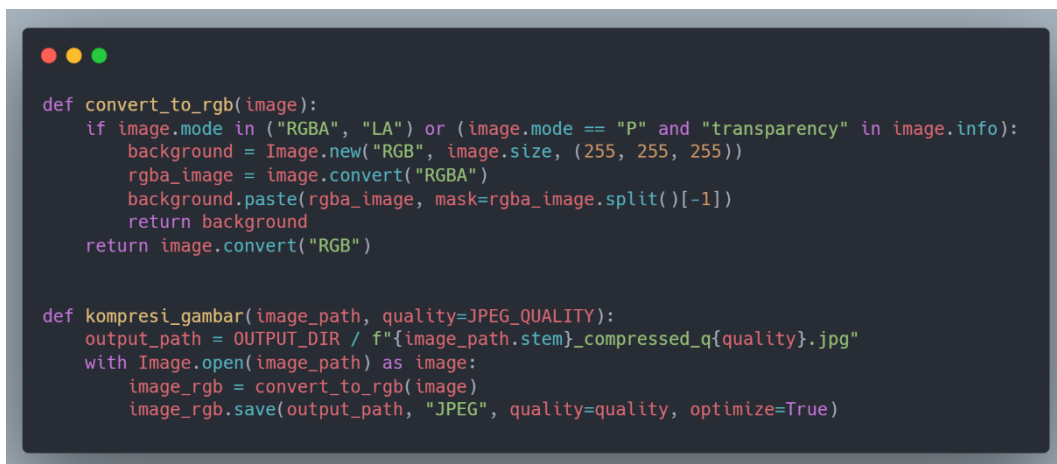
Penentuan *path* dilakukan secara dinamis menggunakan *Path.cwd().resolve()*. Program juga memeriksa apakah eksekusi dilakukan dari *folder notebook* atau dari

root project. Dengan cara ini, program tetap dapat menemukan struktur *folder* yang benar meskipun dijalankan dari lokasi kerja yang berbeda. Selain itu, *folder output* dan *folder results* dibuat otomatis jika belum tersedia.

3.4.3 Fungsi Kompresi Gambar

Fungsi kompresi gambar menjadi bagian utama dari implementasi program. Sebelum gambar disimpan sebagai *JPEG*, program memastikan bahwa gambar berada pada mode *RGB*. Hal ini diperlukan karena format *JPEG* tidak mendukung kanal transparansi. Oleh sebab itu, gambar dengan mode transparan seperti *RGBA*, *LA*, atau *P* dengan informasi transparansi dikonversi terlebih dahulu dengan menambahkan latar belakang putih.

Setelah proses konversi selesai, gambar disimpan ulang sebagai *JPEG* menggunakan *Image.save()*. Pada proses penyimpanan ini, parameter *quality* digunakan untuk mengatur tingkat kualitas kompresi, sedangkan *optimize=True* digunakan untuk mengoptimalkan hasil penyimpanan. Nama *file output* dibuat secara otomatis dengan menambahkan keterangan *_compressed_q{quality}.jpg* pada nama dasar *file* asli.



```
def convert_to_rgb(image):
    if image.mode in ("RGBA", "LA") or (image.mode == "P" and "transparency" in image.info):
        background = Image.new("RGB", image.size, (255, 255, 255))
        rgba_image = image.convert("RGBA")
        background.paste(rgba_image, mask=rgba_image.split()[-1])
        return background
    return image.convert("RGB")

def kompresi_gambar(image_path, quality=JPEG_QUALITY):
    output_path = OUTPUT_DIR / f"{image_path.stem}_compressed_q{quality}.jpg"
    with Image.open(image_path) as image:
        image_rgb = convert_to_rgb(image)
        image_rgb.save(output_path, "JPEG", quality=quality, optimize=True)
```

Gambar 3.8 Fungsi Konversi *RGB* dan Kompresi *JPEG*

Cuplikan tersebut memperlihatkan dua bagian penting, yaitu fungsi *convert_to_rgb()* dan bagian awal fungsi *kompresi_gambar()*. Fungsi *convert_to_rgb()* memastikan gambar siap disimpan sebagai *JPEG*, sedangkan

fungsi *kompresi_gambar()* membuka gambar, melakukan konversi, dan menyimpan gambar hasil kompresi. Dengan demikian, proses kompresi dilakukan melalui fitur internal *Pillow*, bukan melalui implementasi manual algoritma *JPEG*.

Setelah gambar disimpan, fungsi utama juga menghitung ukuran *file* asli, ukuran *file* hasil kompresi, rasio kompresi, *MSE*, dan *PSNR*. Seluruh hasil tersebut dikembalikan dalam bentuk *dictionary* agar mudah dikumpulkan menjadi data tabel pada tahap berikutnya.

```
ukuran_awal = ukuran_kb(image_path)
ukuran_kompresi = ukuran_kb(output_path)
rasio_kompresi = (1 - (ukuran_kompresi / ukuran_awal)) * 100 if ukuran_awal else 0

mse, psnr = hitung_mse_psnr(image_path, output_path)

return {
    "Nama File": image_path.name,
    "File Hasil": output_path.name,
    "Ukuran Awal": format_kb(ukuran_awal),
    "Ukuran Hasil Kompresi": format_kb(ukuran_kompresi),
    "Rasio Kompresi (%)": round(rasio_kompresi, 2),
    "MSE": round(mse, 2),
    "PSNR (dB)": "inf" if math.isinf(psnr) else round(psnr, 2),
}
```

Gambar 3.9 Struktur Hasil Fungsi Kompresi

Cuplikan tersebut menunjukkan bahwa setiap proses kompresi menghasilkan data yang langsung siap digunakan untuk pengujian. Data yang dikembalikan mencakup nama *file*, ukuran awal, ukuran hasil kompresi, rasio kompresi, *MSE*, dan *PSNR*. Bagian ini hanya menjelaskan struktur hasil yang dihasilkan oleh fungsi, sedangkan angka hasil dan interpretasinya dibahas pada subbab hasil pengujian.

3.4.4 Fungsi Perhitungan *MSE* dan *PSNR*

Perhitungan *MSE* dan *PSNR* dilakukan melalui fungsi khusus bernama *hitung_mse_psnr()*. Fungsi ini menerima dua *path*, yaitu *path* gambar asli dan *path* gambar hasil kompresi. Kedua gambar dibuka menggunakan *Pillow*, dikonversi ke mode *RGB*, lalu diubah menjadi *array NumPy* dengan tipe data *float64* agar perhitungan selisih *pixel* dapat dilakukan secara numerik.

MSE dihitung dari rata-rata kuadrat selisih nilai *pixel* antara gambar asli dan gambar hasil kompresi. Setelah nilai *MSE* diperoleh, *PSNR* dihitung menggunakan nilai tersebut. Jika nilai *MSE* sama dengan nol, maka *PSNR* dianggap tak hingga karena tidak terdapat perbedaan antara gambar asli dan gambar hasil kompresi.

A screenshot of a code editor window with a dark background and light-colored text. The code defines a function named `hitung_mse_psnr` that takes two arguments: `original_path` and `compressed_path`. Inside the function, it uses `Image.open` to load the images, converts them to RGB, and then to NumPy arrays using `np.asarray` with `dtype=np.float64`. It then calculates the Mean Squared Error (*MSE*) as the mean of the squared differences between the original and compressed arrays. The Peak Signal-to-Noise Ratio (*PSNR*) is calculated using the formula $20 \cdot \log_{10}(255.0 / \sqrt{mse})$, with a fallback to infinity if *MSE* is zero. The function returns both *MSE* and *PSNR*.

```
def hitung_mse_psnr(original_path, compressed_path):  
    with Image.open(original_path) as original_image, Image.open(compressed_path) as compressed_image:  
        original_rgb = convert_to_rgb(original_image)  
        compressed_rgb = convert_to_rgb(compressed_image)  
  
        original_array = np.asarray(original_rgb, dtype=np.float64)  
        compressed_array = np.asarray(compressed_rgb, dtype=np.float64)  
  
        mse = np.mean((original_array - compressed_array) ** 2)  
        psnr = math.inf if mse == 0 else 20 * math.log10(255.0 / math.sqrt(mse))  
    return mse, psnr
```

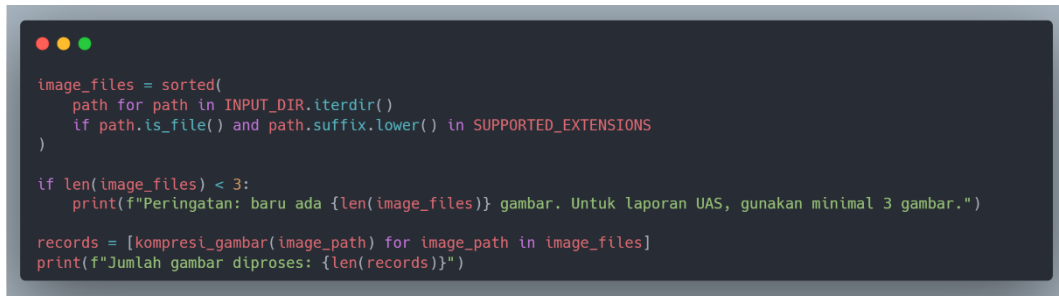
Gambar 3.10 Fungsi Perhitungan *MSE* dan *PSNR*

Cuplikan tersebut menunjukkan bahwa perhitungan kualitas gambar dilakukan dengan membandingkan gambar asli dan gambar hasil kompresi pada tingkat *pixel*. Penggunaan *NumPy* membantu proses perhitungan karena data gambar dapat diproses sebagai *array* numerik. Meskipun demikian, *MSE* dan *PSNR* tetap dipahami sebagai metrik kuantitatif, sehingga interpretasinya perlu dilengkapi dengan perbandingan visual pada bagian hasil pembahasan.

3.4.5 Proses *Batch Compression*

Proses *batch compression* digunakan agar seluruh gambar pada *folder input* dapat diproses secara otomatis. Program memindai isi *folder input*, memfilter *file* berdasarkan ekstensi yang didukung, mengurutkan *file* secara alfabetis, kemudian menjalankan fungsi *kompresi_gambar()* untuk setiap gambar. Hasil dari setiap proses kompresi dikumpulkan ke dalam variabel *records*.

Pendekatan ini membuat proses kompresi lebih efisien karena pengguna tidak perlu menjalankan fungsi kompresi satu per satu untuk setiap gambar. Seluruh gambar yang memenuhi kriteria ekstensi dapat diproses dalam satu alur eksekusi. Namun, proses ini masih berjalan secara sekuensial, sehingga gambar diproses satu per satu dan belum menggunakan *parallel processing*.



```

image_files = sorted(
    path for path in INPUT_DIR.iterdir()
    if path.is_file() and path.suffix.lower() in SUPPORTED_EXTENSIONS
)

if len(image_files) < 3:
    print(f"Peringatan: baru ada {len(image_files)} gambar. Untuk laporan UAS, gunakan minimal 3 gambar.")

records = [kompresi_gambar(image_path) for image_path in image_files]
print(f"Jumlah gambar diproses: {len(records)}")

```

Gambar 3.11 Proses *Batch Compression*

Cuplikan tersebut menunjukkan mekanisme pemrosesan gambar secara *batch*. Program membaca seluruh *file* yang berada pada *folder input*, menyaring *file* berdasarkan ekstensi, kemudian memproses setiap gambar menggunakan fungsi kompresi. Hasil eksekusi menunjukkan bahwa program berhasil memproses tiga gambar.

3.5 Data dan Parameter Pengujian

Data dan parameter pengujian merupakan bagian penting untuk menjelaskan kondisi awal sebelum proses kompresi dilakukan. Pada bagian ini, pengujian menggunakan tiga gambar *JPEG* dengan karakteristik berbeda, yaitu gambar manusia, pemandangan, dan teks. Ketiga gambar tersebut disimpan pada *folder data/input/original/* dan diproses menggunakan parameter kompresi yang sama agar hasil pengujian dapat dibandingkan secara konsisten .

Parameter utama yang digunakan dalam pengujian adalah *quality* pada format *JPEG*. Nilai *quality* menentukan tingkat kualitas gambar saat disimpan ulang menggunakan *Pillow*. Selain itu, program juga menggunakan parameter *optimize=True* pada proses penyimpanan *JPEG*. Dengan parameter tersebut, seluruh gambar *input* diproses menggunakan konfigurasi yang sama, sehingga perbedaan hasil kompresi lebih dipengaruhi oleh karakteristik gambar.

Hasil dari proses kompresi disimpan dalam format *JPEG* pada *folder output data/output/compressed/*. Selain gambar hasil kompresi, program juga menyimpan hasil pengujian dalam bentuk *file CSV* dan *Markdown* pada *folder data/results/*. Dengan demikian, bagian ini menjelaskan tiga hal utama, yaitu data gambar uji, parameter *quality JPEG*, dan format *output* kompresi yang digunakan.

3.5.1 Data Gambar Uji

Data gambar uji terdiri dari tiga *file* gambar berformat *JPEG*. Ketiga gambar tersebut memiliki karakteristik visual yang berbeda, yaitu gambar manusia, gambar pemandangan, dan gambar teks. Pemilihan jenis gambar yang berbeda dilakukan agar proses kompresi dapat diamati pada beberapa bentuk konten visual, meskipun jumlah gambar yang digunakan masih terbatas.

Seluruh gambar *input* berada dalam mode *RGB* dan memiliki resolusi tinggi. Gambar *01-manusia.jpg* memiliki ukuran 4.274,61 KB dengan dimensi 5.231×3.487 *pixel*. Gambar *02-pemandangan.jpg* memiliki ukuran 3.903,40 KB dengan dimensi 5.760×3.840 *pixel*. Sementara itu, gambar *03-teks.jpg* memiliki ukuran 4.748,37 KB dengan dimensi 5.595×3.980 *pixel*. Total ukuran gambar *input* yang digunakan dalam pengujian adalah 12.926,38 KB.

Ringkasan data gambar uji dapat dilihat pada tabel berikut.

Tabel 3.2 Data Gambar Uji

Nama File	Ukuran Asli	Dimensi	Mode	Format
<i>01-manusia.jpg</i>	4.274,61 KB	5.231×3.487 <i>pixel</i>	<i>RGB</i>	<i>JPEG</i>
<i>02-pemandangan.jpg</i>	3.903,40 KB	5.760×3.840 <i>pixel</i>	<i>RGB</i>	<i>JPEG</i>
<i>03-teks.jpg</i>	4.748,37 KB	5.595×3.980 <i>pixel</i>	<i>RGB</i>	<i>JPEG</i>
Total	12.926,38 KB	-	-	-

Tabel tersebut menunjukkan bahwa seluruh data uji memiliki ukuran *file* yang cukup besar dan resolusi tinggi. Data ini menjadi dasar untuk melihat perubahan ukuran setelah proses kompresi dilakukan. Namun, tiga gambar tersebut tidak dapat dianggap mewakili seluruh jenis gambar digital, sehingga hasil pengujian tetap perlu dibaca dalam ruang lingkup gambar yang digunakan pada laporan ini.

3.5.2 Parameter Quality JPEG

Parameter utama yang digunakan dalam proses kompresi adalah *JPEG_QUALITY* dengan nilai 60. Nilai ini didefinisikan sebagai konstanta pada program, kemudian

digunakan sebagai nilai bawaan pada fungsi kompresi gambar. Dengan cara ini, seluruh gambar *input* diproses menggunakan nilai *quality* yang sama.

Selain parameter *quality*, proses penyimpanan gambar juga menggunakan opsi *optimize=True*. Opsi ini diterapkan ketika gambar disimpan ulang dalam format *JPEG* menggunakan *Pillow*. Pada implementasi program, parameter *quality* dan *optimize* diterapkan langsung pada fungsi penyimpanan gambar, sehingga hasil *output* mengikuti konfigurasi yang telah ditentukan.



```

JPEG_QUALITY = 60

def kompresi_gambar(image_path, quality=JPEG_QUALITY):
    ...
    image_rgb.save(output_path, "JPEG", quality=quality, optimize=True)

```

Gambar 3.12 Parameter *Quality JPEG*

Cuplikan tersebut menunjukkan bahwa nilai *quality* sebesar 60 digunakan sebagai parameter utama dalam proses kompresi. Nilai tersebut diterapkan secara konsisten pada seluruh gambar uji. Namun, pengujian ini hanya menggunakan satu tingkat *quality*, sehingga laporan ini tidak dapat menyimpulkan pengaruh variasi *quality* terhadap ukuran *file* atau kualitas gambar secara menyeluruh.

3.5.3 Format *Output* Kompresi

Format *output* kompresi yang digunakan adalah *JPEG*. Setiap gambar hasil kompresi disimpan pada *folder data/output/compressed/* dengan pola penamaan yang konsisten. Pola penamaan tersebut menggunakan nama dasar *file input*, kemudian ditambahkan keterangan *_compressed_q60.jpg*. Dengan pola ini, setiap *file output* dapat langsung dipasangkan dengan *file input* yang sesuai.

Hasil kompresi terdiri dari tiga *file JPEG*. Gambar *01-manusia_compressed_q60.jpg* memiliki ukuran 1.863,69 KB dengan dimensi 5.231 × 3.487 *pixel*. Gambar *02-pemandangan_compressed_q60.jpg* memiliki ukuran 2.195,22 KB dengan dimensi 5.760 × 3.840 *pixel*. Gambar *03-*

teks_compressed_q60.jpg memiliki ukuran 2.666,65 KB dengan dimensi 5.595×3.980 pixel. Total ukuran *file output* adalah 6.725,56 KB.

Perbandingan ukuran *input* dan *output* secara absolut dapat dilihat pada tabel berikut.

Tabel 3.3 Perbandingan Ukuran *Input* dan *Output*

Nama Gambar	Ukuran <i>Input</i>	Ukuran <i>Output</i>	Pengurangan
01-manusia.jpg	4.274,61 KB	1.863,69 KB	2.410,92 KB
02-pemandangan.jpg	3.903,40 KB	2.195,22 KB	1.708,18 KB
03-teks.jpg	4.748,37 KB	2.666,65 KB	2.081,72 KB
Total	12.926,38 KB	6.725,56 KB	6.200,82 KB

Tabel tersebut menunjukkan bahwa seluruh gambar hasil kompresi memiliki ukuran *file* lebih kecil dibandingkan gambar aslinya. Meskipun demikian, bagian ini hanya menampilkan pengurangan ukuran secara absolut. Pembahasan mengenai rasio kompresi, *MSE*, *PSNR*, dan interpretasi kualitas gambar dijelaskan pada subbab hasil pengujian agar alur pembahasan tetap runtut.

Selain gambar hasil kompresi, program juga menghasilkan dua *file* hasil pengujian, yaitu *hasil_pengujian.csv* dan *hasil_pengujian.md*. *File CSV* digunakan untuk menyimpan data dalam bentuk tabel yang dapat diolah kembali, sedangkan *file Markdown* digunakan untuk menyajikan hasil dalam format teks. Kedua *file* tersebut disimpan pada *folder data/results/* dan berisi data hasil pengujian yang sama dalam format berbeda.

3.6 Metrik Evaluasi

Metrik evaluasi digunakan untuk menilai hasil kompresi gambar dari sisi ukuran *file* dan perubahan kualitas gambar. Pada program ini, terdapat tiga metrik utama yang dihitung, yaitu rasio kompresi, *Mean Squared Error* atau *MSE*, dan *Peak Signal-to-Noise Ratio* atau *PSNR*. Seluruh metrik tersebut dihitung otomatis pada fungsi *kompresi_gambar()* dan hasilnya disimpan dalam *file data/results/hasil_pengujian.csv*.

Rasio kompresi digunakan untuk mengetahui seberapa besar ukuran *file output* berkurang dibandingkan dengan ukuran *file input*. Sementara itu, *MSE* dan *PSNR* digunakan untuk mengukur perubahan kualitas gambar secara kuantitatif. *MSE* melihat rata-rata selisih kuadrat nilai *pixel*, sedangkan *PSNR* mengukur tingkat galat kompresi dalam satuan *decibel* atau *dB*.

Selain metrik numerik, program juga menyediakan evaluasi kualitas visual melalui tampilan gambar asli dan gambar hasil kompresi secara berdampingan. Evaluasi visual ini ditampilkan di dalam *notebook* menggunakan *Matplotlib*. Namun, evaluasi visual masih bersifat manual karena tidak terdapat sistem penilaian atau *scoring* visual secara otomatis.

3.6.1 Rasio Kompresi

Rasio kompresi digunakan untuk menunjukkan persentase pengurangan ukuran *file* setelah gambar dikompresi. Perhitungan dilakukan dengan membandingkan ukuran *file input* dan ukuran *file output*. Semakin besar nilai rasio kompresi, semakin besar pula pengurangan ukuran *file* yang dihasilkan dari proses kompresi.

Pada program ini, ukuran *file* dihitung dalam satuan KB menggunakan fungsi *ukuran_kb()*. Setelah ukuran *file input* dan ukuran *file output* diperoleh, rasio kompresi dihitung menggunakan perbandingan antara keduanya. Cuplikan kode berikut menunjukkan bagian perhitungan rasio kompresi pada program.

A screenshot of a code editor window with a dark background and light-colored text. The code is written in Python and calculates the compression ratio. It starts by defining 'ukuran_awal' as the size of the input image file, then 'ukuran_kompresi' as the size of the output compressed file. Finally, it calculates 'rasio_kompresi' as a percentage reduction from the original size, using a conditional expression to handle division by zero.

```
ukuran_awal = ukuran_kb(image_path)
ukuran_kompresi = ukuran_kb(output_path)
rasio_kompresi = (1 - (ukuran_kompresi / ukuran_awal)) * 100 if ukuran_awal else 0
```

Gambar 3.13 Perhitungan Rasio Kompresi

Cuplikan tersebut menunjukkan bahwa rasio kompresi dihitung sebagai persentase pengurangan ukuran *file*. Nilai rasio kemudian digunakan untuk melihat efektivitas kompresi dari sisi ukuran penyimpanan. Metrik ini hanya menjelaskan penurunan

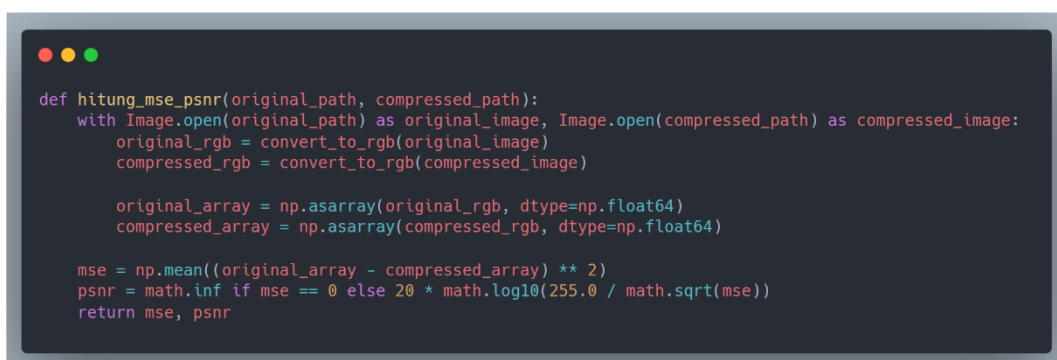
ukuran *file*, sehingga tidak dapat digunakan secara langsung untuk menyimpulkan kualitas visual gambar.

Hasil perhitungan menunjukkan bahwa gambar *01-manusia.jpg* memiliki rasio kompresi tertinggi, yaitu 56,40%. Gambar *02-pemandangan.jpg* memiliki rasio kompresi 43,76%, sedangkan gambar *03-teks.jpg* memiliki rasio kompresi 43,84%. Rata-rata rasio kompresi dari ketiga gambar adalah 48,00%. Perbedaan rasio ini menunjukkan bahwa setiap gambar memberikan hasil pengurangan ukuran *file* yang berbeda meskipun menggunakan parameter *quality* yang sama.

3.6.2 Mean Squared Error (MSE)

Mean Squared Error atau *MSE* digunakan untuk mengukur rata-rata kuadrat selisih nilai *pixel* antara gambar asli dan gambar hasil kompresi. Nilai *MSE* yang lebih kecil menunjukkan bahwa perbedaan nilai *pixel* antara kedua gambar lebih kecil. Sebaliknya, nilai *MSE* yang lebih besar menunjukkan adanya perbedaan nilai *pixel* yang lebih tinggi.

Pada implementasi program, gambar asli dan gambar hasil kompresi terlebih dahulu dibuka menggunakan *Pillow*. Kedua gambar kemudian dikonversi ke mode *RGB* dan diubah menjadi *array NumPy* dengan tipe data *float64*. Setelah itu, program menghitung selisih nilai *pixel* antara gambar asli dan gambar hasil kompresi.

A screenshot of a code editor window with a dark background and light-colored text. The code is a Python function named `hitung_mse_psnr` that takes two paths as input: `original_path` and `compressed_path`. It uses the `Pillow` library to open both images, convert them to RGB, and then to NumPy arrays of type `float64`. The function calculates the Mean Squared Error (MSE) as the mean of the squared differences between the original and compressed arrays. It then calculates the Peak Signal-to-Noise Ratio (PSNR) based on the MSE, using a formula that involves a logarithm. The function returns both the MSE and PSNR values.

```
def hitung_mse_psnr(original_path, compressed_path):
    with Image.open(original_path) as original_image, Image.open(compressed_path) as compressed_image:
        original_rgb = convert_to_rgb(original_image)
        compressed_rgb = convert_to_rgb(compressed_image)

        original_array = np.asarray(original_rgb, dtype=np.float64)
        compressed_array = np.asarray(compressed_rgb, dtype=np.float64)

        mse = np.mean((original_array - compressed_array) ** 2)
        psnr = math.inf if mse == 0 else 20 * math.log10(255.0 / math.sqrt(mse))
    return mse, psnr
```

Gambar 3.14 Perhitungan *MSE* dan *PSNR*

Cuplikan tersebut menunjukkan bahwa *MSE* dihitung dari rata-rata kuadrat selisih *array pixel* antara gambar asli dan gambar hasil kompresi. Penggunaan *array NumPy* membuat proses perhitungan dapat dilakukan secara langsung pada data

numerik gambar. Namun, *MSE* tetap memiliki keterbatasan karena hanya menghitung perbedaan nilai *pixel* secara global dan tidak selalu mewakili persepsi visual manusia secara penuh.

Hasil pengujian menunjukkan bahwa gambar *02-pemandangan.jpg* memiliki nilai *MSE* terendah, yaitu 17,56. Gambar *03-teks.jpg* memiliki nilai *MSE* sebesar 27,05, sedangkan gambar *01-manusia.jpg* memiliki nilai *MSE* tertinggi, yaitu 34,94. Rata-rata nilai *MSE* dari ketiga gambar adalah 26,52.

3.6.3 Peak Signal-to-Noise Ratio (PSNR)

Peak Signal-to-Noise Ratio atau *PSNR* digunakan untuk mengukur kualitas gambar hasil kompresi berdasarkan nilai galat yang diperoleh dari *MSE*. Nilai *PSNR* dinyatakan dalam satuan *decibel* atau *dB*. Secara umum, semakin tinggi nilai *PSNR*, semakin kecil galat antara gambar asli dan gambar hasil kompresi.

Pada program ini, nilai *PSNR* dihitung dari nilai *MSE* menggunakan fungsi matematis *math.log10()* dan *math.sqrt()*. Jika nilai *MSE* sama dengan nol, maka nilai *PSNR* dianggap tak hingga karena tidak terdapat perbedaan nilai *pixel* antara gambar asli dan gambar hasil kompresi. Dalam pengujian ini, seluruh gambar memiliki nilai *MSE* lebih dari nol sehingga nilai *PSNR* dapat dihitung dalam satuan *dB*.

Ringkasan hasil metrik evaluasi dapat dilihat pada tabel berikut.

Tabel 3.4 Hasil Metrik Evaluasi Kompresi

Nama Gambar	Rasio Kompresi (%)	<i>MSE</i>	<i>PSNR (dB)</i>
<i>01-manusia.jpg</i>	56,4	34,9	32,7
<i>02-pemandangan.jpg</i>	43,76	17,6	35,69
<i>03-teks.jpg</i>	43,84	27,1	33,81
Rata-rata	48	26,5	34,07

Tabel tersebut menunjukkan bahwa setiap gambar memiliki karakteristik hasil yang berbeda. Gambar *01-manusia.jpg* memiliki rasio kompresi tertinggi, tetapi juga

memiliki nilai *MSE* tertinggi dan *PSNR* terendah. Sebaliknya, gambar *02-pemandangan.jpg* memiliki rasio kompresi terendah, tetapi menghasilkan nilai *MSE* terendah dan *PSNR* tertinggi. Namun, hubungan antar metrik tersebut tidak boleh disimpulkan secara berlebihan karena pengujian hanya menggunakan satu nilai *quality*.

Hasil perhitungan menunjukkan bahwa gambar *02-pemandangan.jpg* memiliki nilai *PSNR* tertinggi, yaitu 35,69 *dB*. Gambar *03-teks.jpg* memiliki nilai *PSNR* sebesar 33,81 *dB*, sedangkan gambar *01-manusia.jpg* memiliki nilai *PSNR* terendah, yaitu 32,70 *dB*. Rata-rata nilai *PSNR* dari ketiga gambar adalah 34,07 *dB*. Nilai ini menunjukkan bahwa seluruh gambar memiliki hasil pengukuran *PSNR* yang berada di atas 30 *dB*, tetapi kesimpulan kualitas visual tetap perlu dikaitkan dengan pengamatan manual.

3.7 Output Program

Output program merupakan hasil akhir yang diperoleh setelah seluruh proses kompresi dan pengujian dijalankan. Pada bagian ini, program menghasilkan tiga jenis *output*, yaitu gambar *JPEG* hasil kompresi, tabel hasil pengujian, dan *preview* perbandingan visual antara gambar asli dan gambar hasil kompresi. Seluruh *output* tersebut dihasilkan melalui *notebook* yang telah dijalankan secara berurutan .

Gambar hasil kompresi disimpan pada *folder data/output/compressed/*, sedangkan tabel hasil pengujian disimpan pada *folder data/results/* dalam dua format, yaitu *CSV* dan *Markdown*. Selain itu, program juga menampilkan *preview* visual secara *inline* di dalam *Jupyter Notebook* menggunakan *Matplotlib*. Dengan adanya tiga bentuk *output* tersebut, hasil kompresi dapat dilihat dari sisi *file* gambar, data pengujian, dan tampilan visual.

Ringkasan jenis *output* yang dihasilkan program dapat dilihat pada tabel berikut.

Tabel 3.5 Ringkasan *Output* Program

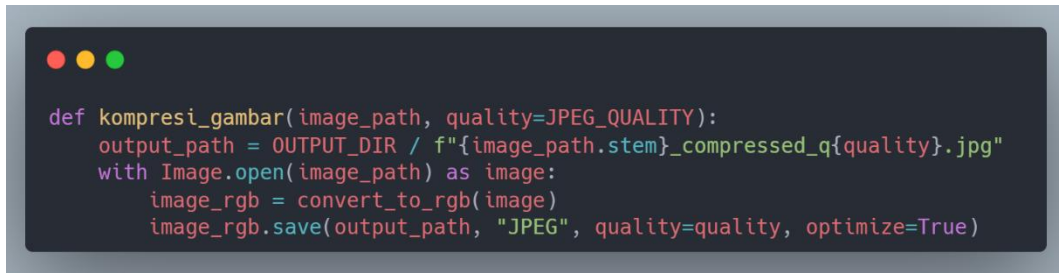
Jenis <i>Output</i>	Lokasi	Jumlah <i>File</i>	Format	Status
Gambar hasil kompresi	<i>data/output/compressed/</i>	3	<i>JPEG</i>	Tersedia
Tabel hasil pengujian	<i>data/results/</i>	1	<i>CSV</i>	Tersedia
Tabel hasil pengujian	<i>data/results/</i>	1	<i>Markdown</i>	Tersedia
<i>Preview visual</i>	<i>notebook</i>	-	<i>Plot inline</i>	Tersedia

Tabel tersebut menunjukkan bahwa program menghasilkan *output* yang lengkap untuk kebutuhan pembahasan hasil. Gambar hasil kompresi digunakan untuk melihat *file output* secara langsung, tabel hasil pengujian digunakan untuk memeriksa data numerik, sedangkan *preview visual* digunakan sebagai pendukung evaluasi tampilan gambar.

3.7.1 Gambar Hasil Kompresi

Gambar hasil kompresi merupakan *output* utama dari program. Setiap gambar *input* yang berada pada *folder data/input/original/* diproses dan disimpan kembali sebagai *file JPEG* pada *folder data/output/compressed/*. Proses penyimpanan gambar dilakukan menggunakan *Pillow* dengan format *JPEG*.

Pola penamaan *file output* dibuat secara konsisten dengan menambahkan keterangan *_compressed_q60.jpg* pada nama dasar *file input*. Pola ini memudahkan proses identifikasi karena setiap *file output* dapat langsung dipasangkan dengan gambar aslinya. Selain itu, dimensi gambar hasil kompresi tetap sama dengan gambar asli, sehingga proses yang dilakukan merupakan kompresi *file*, bukan perubahan resolusi atau *resizing*.



```
def kompresi_gambar(image_path, quality=JPEG_QUALITY):
    output_path = OUTPUT_DIR / f"{image_path.stem}_compressed_q{quality}.jpg"
    with Image.open(image_path) as image:
        image_rgb = convert_to_rgb(image)
        image_rgb.save(output_path, "JPEG", quality=quality, optimize=True)
```

Gambar 3.15 Penyimpanan Gambar Hasil Kompresi

Cuplikan tersebut menunjukkan mekanisme penyimpanan gambar hasil kompresi. Gambar dibuka menggunakan *Image.open()*, dikonversi ke mode *RGB*, kemudian disimpan sebagai *JPEG* menggunakan *Image.save()*. Dengan proses ini, program menghasilkan *file output* baru pada *folder output* tanpa mengganti *file input* asli.

Data gambar hasil kompresi dapat dilihat pada tabel berikut.

Tabel 3.6 Data Gambar *Output* Hasil Kompresi

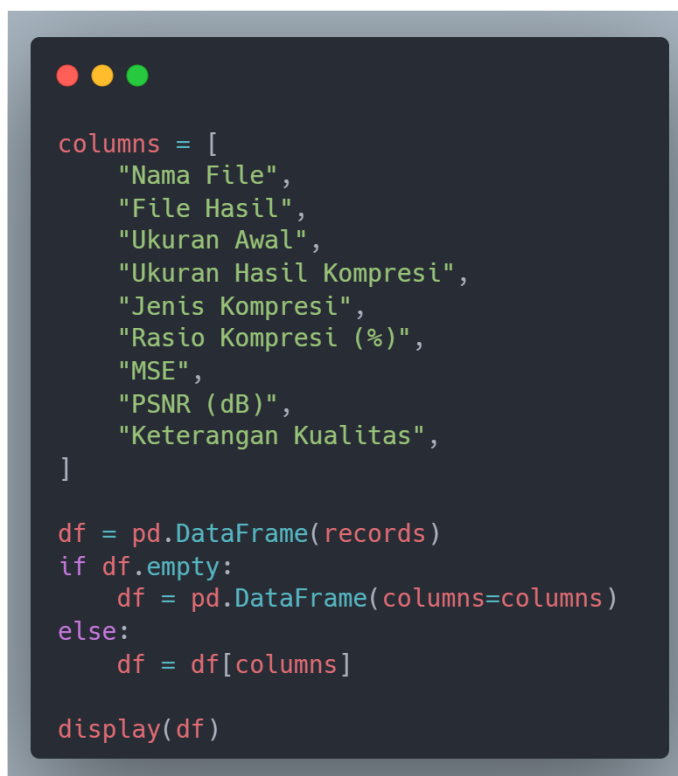
Nama File Output	Ukuran	Dimensi	Mode	Format
01-manusia_compressed_q60.jpg	1.863,69 KB	5.231 × 3.487 pixel	RGB	JPEG
02-pemandangan_compressed_q60.jpg	2.195,22 KB	5.760 × 3.840 pixel	RGB	JPEG
03-teks_compressed_q60.jpg	2.666,65 KB	5.595 × 3.980 pixel	RGB	JPEG
Total	6.725,56 KB	-	-	-

Tabel tersebut memperlihatkan bahwa terdapat tiga *file output* hasil kompresi dengan total ukuran 6.725,56 KB. Seluruh gambar hasil kompresi tetap menggunakan format *JPEG* dan mode *RGB*. Bagian ini hanya menjelaskan keberadaan dan karakteristik dasar *file output*, sedangkan pembahasan rasio kompresi, *MSE*, dan *PSNR* telah ditempatkan pada subbab metrik evaluasi.

3.7.2 Tabel Hasil Pengujian

Selain menghasilkan gambar hasil kompresi, program juga membuat tabel hasil pengujian. Tabel ini dibentuk dari data hasil kompresi yang dikumpulkan ke dalam *list of dict*, kemudian diubah menjadi *DataFrame* menggunakan *Pandas*. Data tersebut memuat informasi utama seperti nama *file*, nama *file hasil*, ukuran awal, ukuran hasil kompresi, jenis kompresi, rasio kompresi, *MSE*, *PSNR*, dan keterangan kualitas.

Tabel hasil pengujian ditampilkan langsung di dalam *notebook* agar dapat diperiksa setelah proses kompresi selesai. Kolom yang digunakan pada tabel sudah ditentukan agar hasil yang ditampilkan tetap terstruktur dan hanya berisi data yang relevan dengan pengujian.



```
columns = [
    "Nama File",
    "File Hasil",
    "Ukuran Awal",
    "Ukuran Hasil Kompresi",
    "Jenis Kompresi",
    "Rasio Kompresi (%)",
    "MSE",
    "PSNR (dB)",
    "Keterangan Kualitas",
]

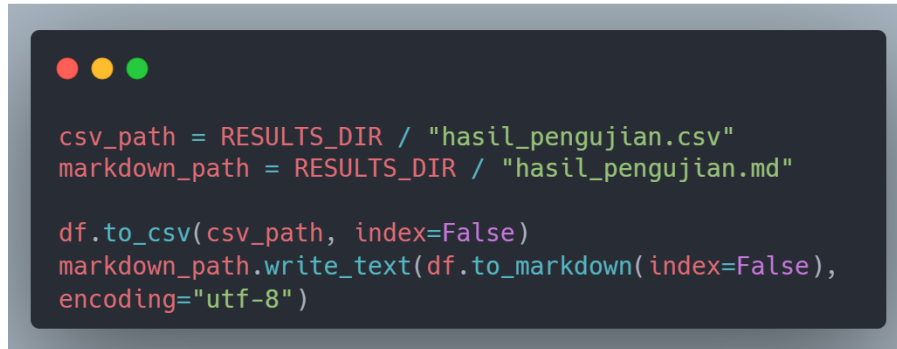
df = pd.DataFrame(records)
if df.empty:
    df = pd.DataFrame(columns=columns)
else:
    df = df[columns]

display(df)
```

Gambar 3.16 Pembuatan Tabel Hasil Pengujian

Cuplikan tersebut menunjukkan bahwa tabel hasil pengujian dibuat dari variabel *records*. Setelah menjadi *DataFrame*, tabel difilter ke dalam sembilan kolom utama, kemudian ditampilkan menggunakan *display()*. Dengan cara ini, hasil

pengujian dapat dilihat langsung di dalam *notebook* sebelum diekspor ke *file* terpisah.



```
csv_path = RESULTS_DIR / "hasil_pengujian.csv"
markdown_path = RESULTS_DIR / "hasil_pengujian.md"

df.to_csv(csv_path, index=False)
markdown_path.write_text(df.to_markdown(index=False),
encoding="utf-8")
```

Gambar 3.17 *Export Tabel Hasil Pengujian ke CSV dan Markdown*

Program juga menyimpan tabel hasil pengujian ke dalam dua format, yaitu *CSV* dan *Markdown*. Kedua format tersebut memiliki isi data yang sama, tetapi digunakan untuk kebutuhan yang berbeda. *CSV* lebih sesuai untuk pengolahan data lanjutan, sedangkan *Markdown* lebih mudah digunakan untuk dokumentasi atau penyalinan tabel ke laporan.

Cuplikan tersebut memperlihatkan bahwa *DataFrame* hasil pengujian diekspor ke dua *file*, yaitu *hasil_pengujian.csv* dan *hasil_pengujian.md*. Kedua *file* tersebut disimpan pada *folder data/results/*. Dengan adanya *file output* ini, hasil pengujian dapat dibuka kembali tanpa harus menjalankan ulang seluruh proses kompresi.

3.7.3 **Preview Perbandingan Gambar**

Preview perbandingan gambar digunakan untuk menampilkan gambar asli dan gambar hasil kompresi secara berdampingan. Visualisasi ini dibuat menggunakan *Matplotlib* di dalam *Jupyter Notebook*. Susunan visual terdiri dari dua kolom, yaitu gambar asli pada sisi kiri dan gambar hasil kompresi pada sisi kanan. Karena terdapat tiga gambar uji, tampilan visual disusun menjadi tiga baris.

Tujuan dari *preview* ini adalah membantu evaluasi visual secara manual. Dengan melihat gambar asli dan gambar hasil kompresi secara berdampingan, perbedaan tampilan dapat diamati secara langsung. Namun, *preview* ini tidak menghasilkan nilai numerik dan tidak menyimpan *plot* sebagai *file* gambar terpisah.

```

if not records:
    display(Markdown("Belum ada gambar untuk ditampilkan."))
else:
    fig, axes = plt.subplots(len(records), 2, figsize=(10, 4 * len(records)))
    if len(records) == 1:
        axes = [axes]

    for row_axes, record in zip(axes, records):
        original_image = Image.open(record["Path Asli"])
        compressed_image = Image.open(record["Path Hasil"])

        row_axes[0].imshow(original_image)
        row_axes[0].set_title(f"Asli: {record['Nama File']}")
        row_axes[0].axis("off")

        row_axes[1].imshow(compressed_image)
        row_axes[1].set_title(f"Kompresi: {record['File Hasil']}")
        row_axes[1].axis("off")

    plt.tight_layout()
    plt.show()

```

Gambar 3.18 Visualisasi Perbandingan Gambar

Cuplikan tersebut menunjukkan bahwa program membuat *subplot* berdasarkan jumlah gambar yang diproses. Setiap pasangan gambar ditampilkan dalam satu baris, dengan gambar asli di sebelah kiri dan gambar hasil kompresi di sebelah kanan. Penggunaan `axis("off")` membuat tampilan lebih fokus pada gambar, sedangkan `plt.tight_layout()` digunakan untuk merapikan tata letak visual.



Gambar 3.19 *Preview Perbandingan Gambar dari Jupyter Notebook*

Evaluasi visual pada bagian ini masih bersifat kualitatif dan manual. Artinya, belum ada *scoring* otomatis, deteksi *artifact*, atau penilaian visual berbasis area tertentu. Oleh karena itu, *preview* perbandingan gambar digunakan sebagai pendukung untuk melihat hasil kompresi, bukan sebagai satu-satunya dasar untuk menyimpulkan kualitas gambar.

4. PENUTUP

4.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, program kompresi gambar menggunakan *Python Pillow* berhasil dibuat dan dijalankan melalui *Jupyter Notebook*. Program mampu membaca gambar dari *folder input*, melakukan kompresi ke format *JPEG*, menyimpan gambar hasil kompresi ke *folder output*, serta menghasilkan tabel pengujian dalam format *CSV* dan *Markdown*.

Kesimpulan yang diperoleh dari laporan ini adalah sebagai berikut.

1. Program berhasil mengompresi tiga gambar uji berformat *JPEG* dengan karakteristik berbeda, yaitu gambar manusia, pemandangan, dan teks. Seluruh gambar diproses menggunakan parameter *quality* 60 dan *optimize=True*.
2. Kompresi yang dilakukan mampu menurunkan ukuran *file* gambar. Total ukuran gambar *input* sebesar 12.926,38 KB berkurang menjadi 6.725,56 KB setelah kompresi, sehingga terjadi pengurangan ukuran sebesar 6.200,82 KB.
3. Berdasarkan metrik evaluasi, rasio kompresi berada pada rentang 43,76% sampai 56,40%, nilai *MSE* berada pada rentang 17,56 sampai 34,94, dan nilai *PSNR* berada pada rentang 32,70 *dB* sampai 35,69 *dB*.
4. Program juga menghasilkan *preview* perbandingan visual antara gambar asli dan gambar hasil kompresi. Namun, evaluasi visual masih bersifat manual karena belum terdapat *scoring* otomatis atau analisis visual berbasis area tertentu.
5. Pengujian ini belum dapat menyimpulkan pengaruh variasi nilai *quality* terhadap ukuran *file* dan kualitas gambar, karena hanya menggunakan satu nilai *quality*, yaitu 60.

Secara keseluruhan, *Python Pillow* dapat digunakan untuk melakukan kompresi gambar *JPEG* secara sederhana, terstruktur, dan mudah diuji. Program tidak hanya menghasilkan gambar hasil kompresi, tetapi juga menyediakan metrik evaluasi yang dapat digunakan untuk membandingkan ukuran *file* dan perubahan kualitas gambar.

4.2 Saran

Berdasarkan hasil yang diperoleh, pengujian selanjutnya disarankan menggunakan beberapa variasi nilai *quality*, misalnya 30, 50, 70, dan 90. Variasi tersebut diperlukan agar hubungan antara nilai *quality*, ukuran *file*, rasio kompresi, *MSE*, dan *PSNR* dapat dianalisis secara lebih jelas.

Selain itu, jumlah dan jenis gambar uji dapat ditambah agar hasil pengujian lebih beragam. Pengujian berikutnya dapat menggunakan gambar ilustrasi, gambar bertekstur kompleks, gambar beresolusi rendah, atau gambar dengan karakteristik warna yang berbeda. Dengan data yang lebih bervariasi, hasil analisis kompresi akan menjadi lebih representatif.

Evaluasi kualitas gambar juga dapat dikembangkan dengan menambahkan metrik lain seperti *Structural Similarity Index Measure* atau *SSIM*. Metrik tersebut dapat digunakan sebagai pelengkap *MSE* dan *PSNR* karena penilaian kualitas gambar tidak selalu cukup dilihat dari perbedaan nilai *pixel* saja.

Program juga dapat dibuat lebih fleksibel dengan menyediakan pengaturan *quality*, *folder input*, dan *folder output* melalui konfigurasi terpisah. Selain itu, hasil *preview* visual sebaiknya disimpan sebagai *file output* agar dapat langsung digunakan sebagai lampiran atau bukti visual dalam laporan.

DAFTAR PUSTAKA

- Adobe. (2024). *Lossy vs Lossless Compression: Differences & Advantages*. Wwww.Adobe.Com.
<https://www.adobe.com/uk/creativecloud/photography/discover/lossy-vs-lossless.htm>
- Cloudinary. (2020). *Image Optimization for Websites*. Cloudinary.Com.
https://cloudinary.com/blog/image_optimization_for_websites_beautiful_pages_that_load_quickly
- Endah Pangesti, W., Widagdo, G., Riana, D., Hadiani, S., & Magister Ilmu Komputer Sekolah Tinggi Ilmu Manajemen dan Informatika Nusa Mandiri Jakarta wwww.nusamandiri.ac.id, P. (2029). IMPLEMENTASI KOMPRESI CITRA DIGITAL DENGAN MEMBANDINGKAN METODE LOSSY DAN LOSSLESS COMPRESSION MENGGUNAKAN MATLAB. *Jurnal Khatulistiwa Informatika*, 8(1).
<https://ejournal.bsi.ac.id/index.php/khatulistiwa/article/view/7759>
- Evident Scientific. (2026). *Digital Imaging Basics: Pixels, Bit Depth & Color Space Models*. Evidentscientific.Com.
<https://evidentscientific.com/en/microscope-resource/knowledge-hub/digital-imaging/digitalimagebasics>
- Gregory K., W. (1992). The JPEG Still Picture Compression Standard. *IEEE Transactions on Consumer Electronics*, 38(1).
<https://ieeexplore.ieee.org/document/125072/>
- Hussain, A. J., Al-Fayadh, A., Radi, N., Dhab, A., & Bahia Abu Dhab, A. (2018). Image Compression Techniques: A Survey in Lossless and Lossy algorithms. *Neurocomputing*.
<https://www.sciencedirect.com/science/article/abs/pii/S0925231218302935>

IBM. (2024). *What is data redundancy?* Wwww.Ibm.Com.
<https://www.ibm.com/think/topics/data-redundancy>

ITU-T. (1993). *TERMINAL EQUIPMENT AND PROTOCOLS FOR TELEMATIC SERVICES INFORMATION TECHNOLOGY-DIGITAL COMPRESSION AND CODING OF CONTINUOUS-TONE STILL IMAGES-REQUIREMENTS AND GUIDELINES Recommendation T.81.*
<https://www.w3.org/Graphics/JPEG/itu-t81.pdf>

John, J. (2021). Discrete Cosine Transform in JPEG Compression. *ArXiv Preprint ArXiv:2102.06968*. <http://arxiv.org/abs/2102.06968>

MathWorks. (2024). *Compute peak signal-to-noise ratio (PSNR) between images.* Wwww.Mathworks.Com.
<https://www.mathworks.com/help/vision/ref/psnr.html>

MDN Web Docs. (2025a). *Image file type and format guide.* Developer.Mozilla.Org. https://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats/Image_types

MDN Web Docs. (2025b). *Lossless compression.* Developer.Mozilla.Org. https://developer.mozilla.org/en-US/docs/Glossary/Lossless_compression

MDN Web Docs. (2025c). *Lossy compression.* Developer.Mozilla.Org. https://developer.mozilla.org/en-US/docs/Glossary/Lossy_compression

MDN Web Docs. (2026). *Image file type and format guide.* Developer.Mozilla.Org. https://developer.mozilla.org/en-US/docs/Web/Media/Guides/Formats/Image_types

Pillow. (2026a). *Image file formats.* Pillow.Readthedocs.i. <https://pillow.readthedocs.io/en/stable/handbook/image-file-formats.html>

Pillow. (2026b). *Image module.* Pillow.Readthedocs.i. <https://pillow.readthedocs.io/en/stable/reference/Image.html>

Pillow. (2026c). *Pillow.* Pypi.Org. <https://pypi.org/project/pillow/>

Pillow. (2026d). *Python Pillow*. Python-Pillow.Github.Io. <https://python-pillow.github.io>

Salomon, D. (2007). *Data Compression: The Complete Reference* (3rd ed.). Springer-Verlag New York, Inc.

Shawahna, A., Haque, Md. E., & Amin, A. (2019). JPEG Image Compression using the Discrete Cosine Transform: An Overview, Applications, and Hardware Implementation. *ArXiv Preprint ArXiv:1912.10789*. <http://arxiv.org/abs/1912.10789>

Web.dev. (2023). *Image performance*. Web.Dev. <https://web.dev/learn/performance/image-performance>

LAMPIRAN

Lampiran 1 *Setup* Kode

```
from pathlib import Path
import math

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Markdown, display
from PIL import Image

PROJECT_ROOT = Path.cwd().resolve()
if PROJECT_ROOT.name == "notebook":
    PROJECT_ROOT = PROJECT_ROOT.parent
elif not (PROJECT_ROOT / "data").exists() and (PROJECT_ROOT.parent / "data").exists():
    PROJECT_ROOT = PROJECT_ROOT.parent

INPUT_DIR = PROJECT_ROOT / "data" / "input" / "original"
OUTPUT_DIR = PROJECT_ROOT / "data" / "output" / "compressed"
RESULTS_DIR = PROJECT_ROOT / "data" / "results"

OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
RESULTS_DIR.mkdir(parents=True, exist_ok=True)

SUPPORTED_EXTENSIONS = {".jpg", ".jpeg", ".png"}
JPEG_QUALITY = 60

print(f"Project root: {PROJECT_ROOT}")
print(f"Folder input: {INPUT_DIR}")
print(f"Folder output: {OUTPUT_DIR}")
```

Lampiran 2 Fungsi Kompresi

```
def ukuran_kb(path):
    return path.stat().st_size / 1024

def format_kb(value):
    return f"{value:,.2f} KB"

def convert_to_rgb(image):
    if image.mode in ("RGBA", "LA") or (image.mode == "P" and "transparency" in image.info):
        background = Image.new("RGB", image.size, (255, 255, 255))
        rgba_image = image.convert("RGBA")
        background.paste(rgba_image, mask=rgba_image.split()[-1])
        return background
    return image.convert("RGB")

def hitung_mse_psnr(original_path, compressed_path):
    with Image.open(original_path) as original_image, Image.open(compressed_path) as compressed_image:
        original_rgb = convert_to_rgb(original_image)
        compressed_rgb = convert_to_rgb(compressed_image)

        original_array = np.asarray(original_rgb, dtype=np.float64)
        compressed_array = np.asarray(compressed_rgb, dtype=np.float64)

        mse = np.mean((original_array - compressed_array) ** 2)
        psnr = math.inf if mse == 0 else 20 * math.log10(255.0 / math.sqrt(mse))
        return mse, psnr

def kompresi_gambar(image_path, quality=JPEG_QUALITY):
    output_path = OUTPUT_DIR / f"{image_path.stem}_compressed_q{quality}.jpg"

    with Image.open(image_path) as image:
        image_rgb = convert_to_rgb(image)
        image_rgb.save(output_path, "JPEG", quality=quality, optimize=True)

    ukuran_awal = ukuran_kb(image_path)
    ukuran_kompresi = ukuran_kb(output_path)
    rasio_kompresi = (1 - (ukuran_kompresi / ukuran_awal)) * 100 if ukuran_awal else 0

    mse, psnr = hitung_mse_psnr(image_path, output_path)

    return {
        "Nama File": image_path.name,
        "File Hasil": output_path.name,
        "Ukuran Awal": format_kb(ukuran_awal),
        "Ukuran Hasil Kompresi": format_kb(ukuran_kompresi),
        "Jenis Kompresi": "JPEG Lossy",
        "Rasio Kompresi (%)": round(rasio_kompresi, 2),
        "MSE": round(mse, 2),
        "PSNR (dB)": "inf" if math.isinf(psnr) else round(psnr, 2),
        "Keterangan Kualitas": "Perlu evaluasi visual manual",
        "Path Asli": str(image_path),
        "Path Hasil": str(output_path),
    }
```

Lampiran 3 Proses Pengujian

```
image_files = sorted(
    path for path in INPUT_DIR.iterdir()
    if path.is_file() and path.suffix.lower() in SUPPORTED_EXTENSIONS
)

if len(image_files) < 3:
    print(f"Peringatan: baru ada {len(image_files)} gambar. Untuk laporan UAS, gunakan minimal 3 gambar.")

records = [kompresi_gambar(image_path) for image_path in image_files]
print(f"Jumlah gambar diproses: {len(records)}")
```

Lampiran 4 Tabel Hasil

```
columns = [
    "Nama File",
    "File Hasil",
    "Ukuran Awal",
    "Ukuran Hasil Kompresi",
    "Jenis Kompresi",
    "Rasio Kompresi (%)",
    "MSE",
    "PSNR (dB)",
    "Keterangan Kualitas",
]

df = pd.DataFrame(records)
if df.empty:
    df = pd.DataFrame(columns=columns)
else:
    df = df[columns]

display(df)
```

Lampiran 5 *Preview Visual*

```
if not records:
    display(Markdown("Belum ada gambar untuk ditampilkan. Tambahkan gambar ke folder `data/input/original/`."))
else:
    fig, axes = plt.subplots(len(records), 2, figsize=(10, 4 * len(records)))
    if len(records) == 1:
        axes = [axes]

    for row_axes, record in zip(axes, records):
        original_image = Image.open(record["Path Asli"])
        compressed_image = Image.open(record["Path Hasil"])

        row_axes[0].imshow(original_image)
        row_axes[0].set_title(f"Asli: {record['Nama File']}")
        row_axes[0].axis("off")

        row_axes[1].imshow(compressed_image)
        row_axes[1].set_title(f"Kompresi: {record['File Hasil']}")
        row_axes[1].axis("off")

    plt.tight_layout()
    plt.show()
```

Lampiran 6 *Export Hasil*

```
csv_path = RESULTS_DIR / "hasil_pengujian.csv"
markdown_path = RESULTS_DIR / "hasil_pengujian.md"

df.to_csv(csv_path, index=False)
markdown_path.write_text(df.to_markdown(index=False), encoding="utf-8")

print(f"CSV tersimpan di: {csv_path}")
print(f"Markdown tersimpan di: {markdown_path}")
```

Lampiran 7 *Tautan Source Code Project*

<https://github.com/zidanindratama/sistem-multimedia-kompresi-gambar>